Module 2

Data Link Layer

Data Link Control

Note

Two main functions of the data link layer are **data link control** and **media access control**.

Data link control deals with the design and procedures for communication between two adjacent nodes.

i.e., node-to-node communication

11-1 FRAMING

The data link layer needs to pack bits into frames, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.

Figure 11.1 *A frame in a character-oriented protocol*

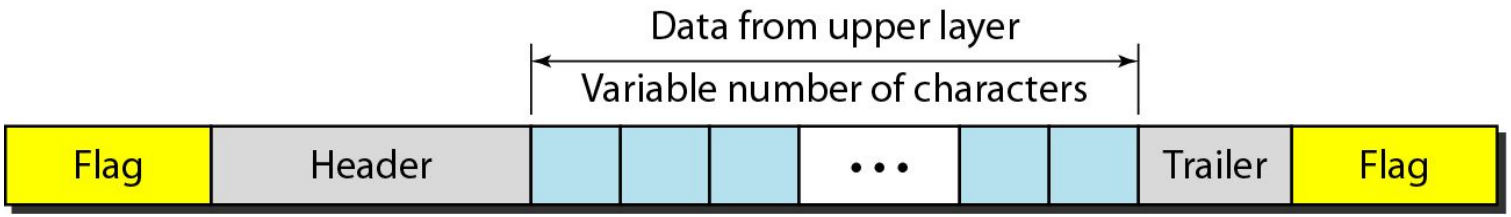
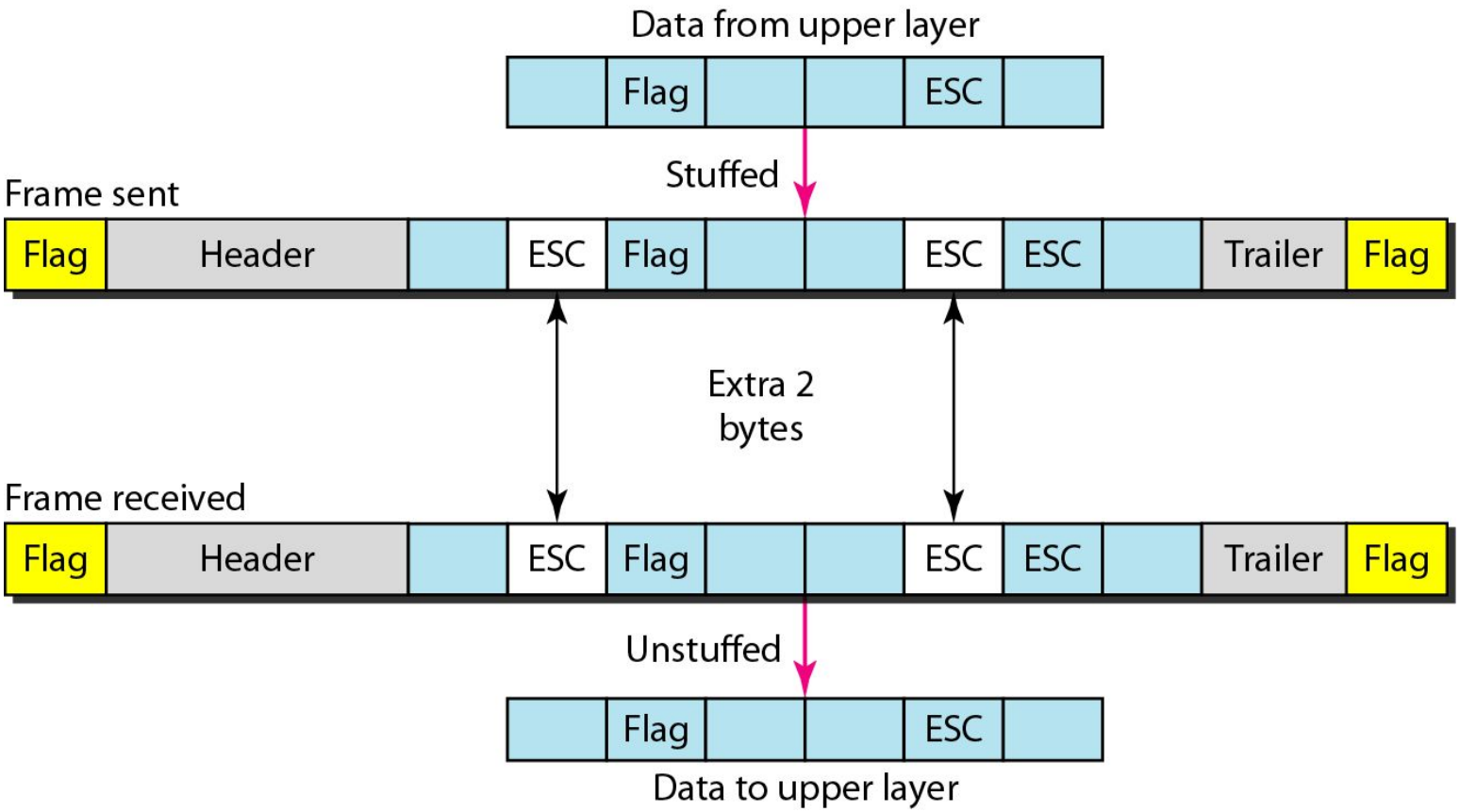


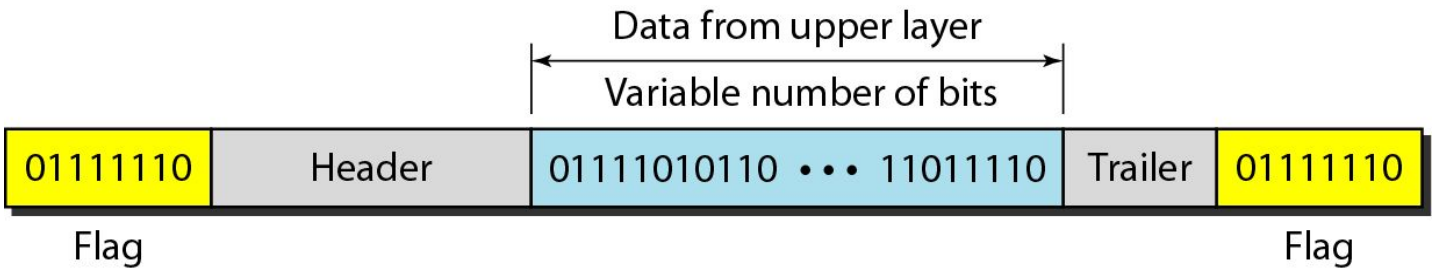
Figure 11.2 *Byte stuffing and unstuffing*



Note

Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text.

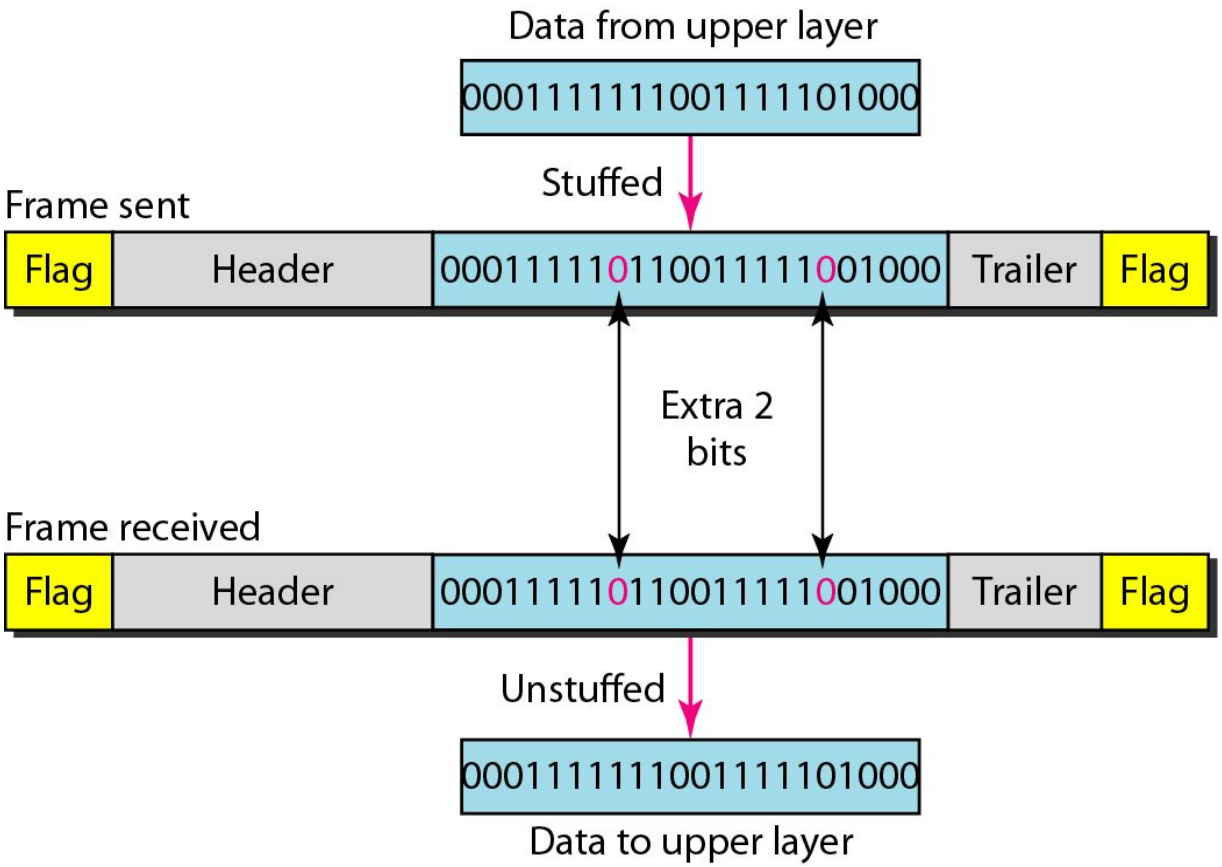
Figure 11.3 *A frame in a bit-oriented protocol*



Note

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

Figure 11.4 *Bit stuffing and unstuffing*



11-2 FLOW AND ERROR CONTROL

The most important responsibilities of the data link layer are flow control and error control. Collectively, these functions are known as data link control.

Topics discussed in this section:

Flow Control
Error Control

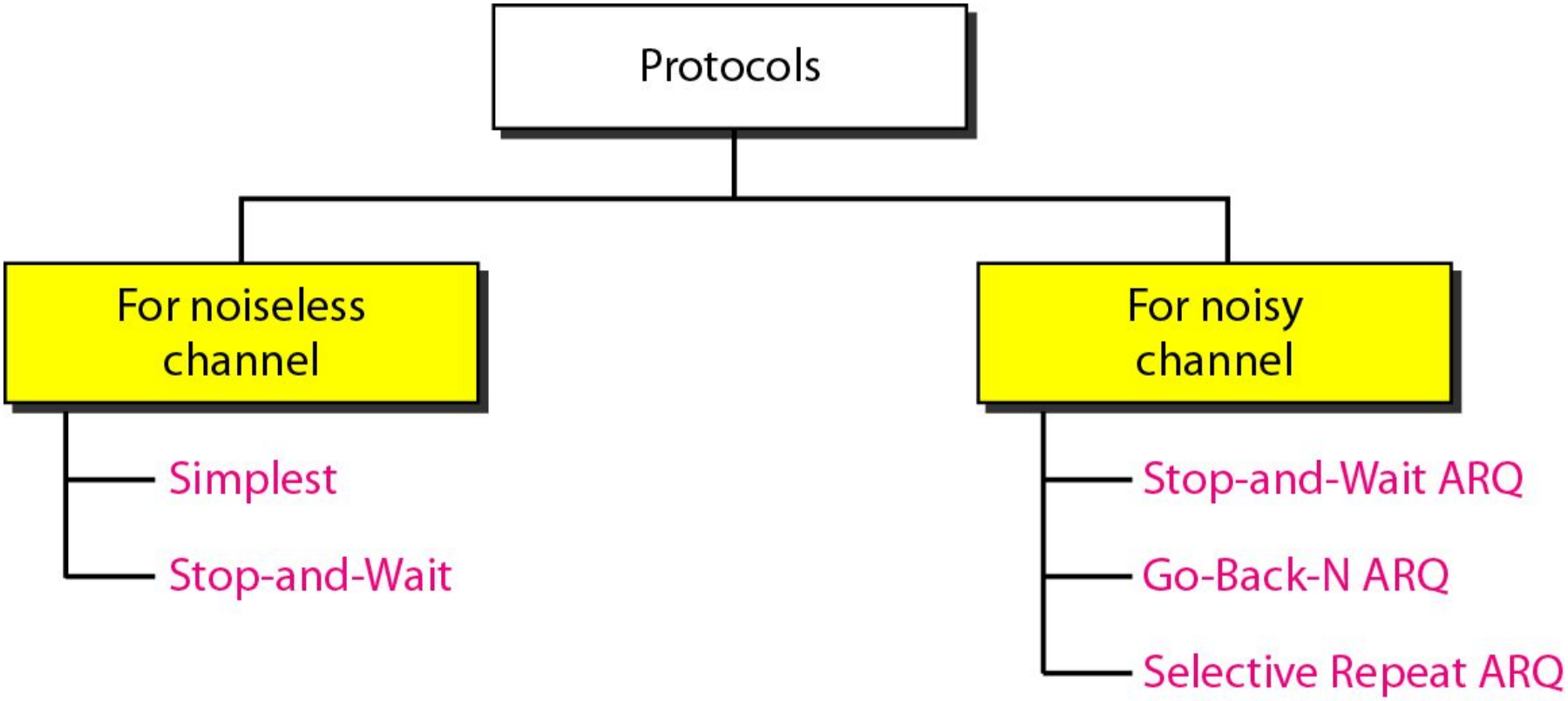
Note

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

Note

Error control in the data link layer is based on automatic repeat request, which is the retransmission of data.

Figure 11.5 *Taxonomy of protocols discussed in this chapter*



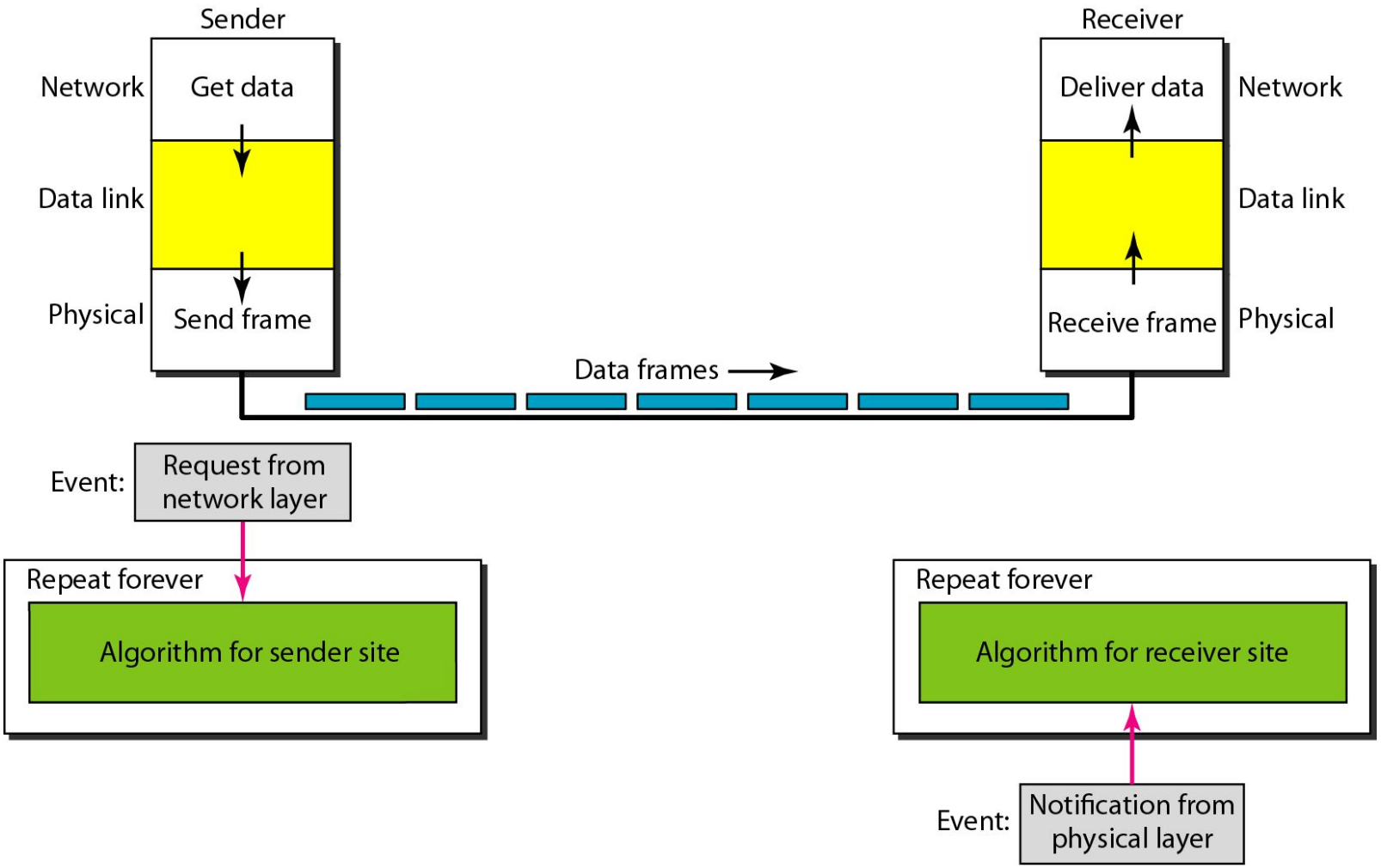
11-4 NOISELESS CHANNELS

Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel.

Topics discussed in this section:

Simplest Protocol
Stop-and-Wait Protocol

Figure 11.6 *The design of the simplest protocol with no flow or error control*



Example 11.1

Figure 11.7 shows an example of communication using this protocol. It is very simple. The sender sends a sequence of frames without even thinking about the receiver. To send three frames, three events occur at the sender site and three events at the receiver site. Note that the data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.

Figure 11.7 *Flow diagram for Example 11.1*

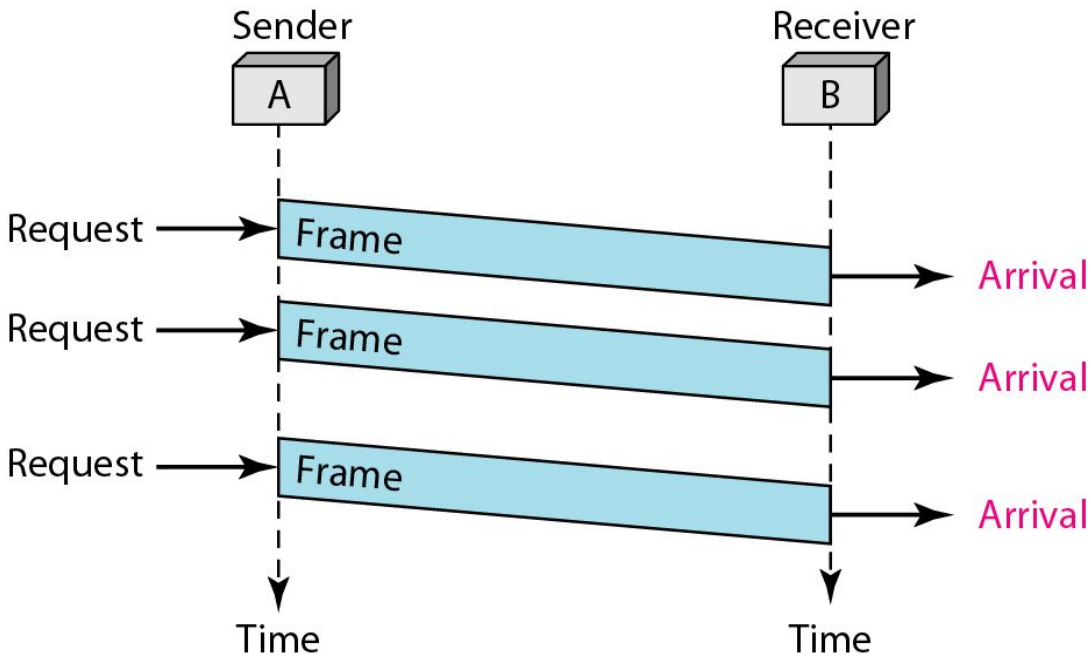
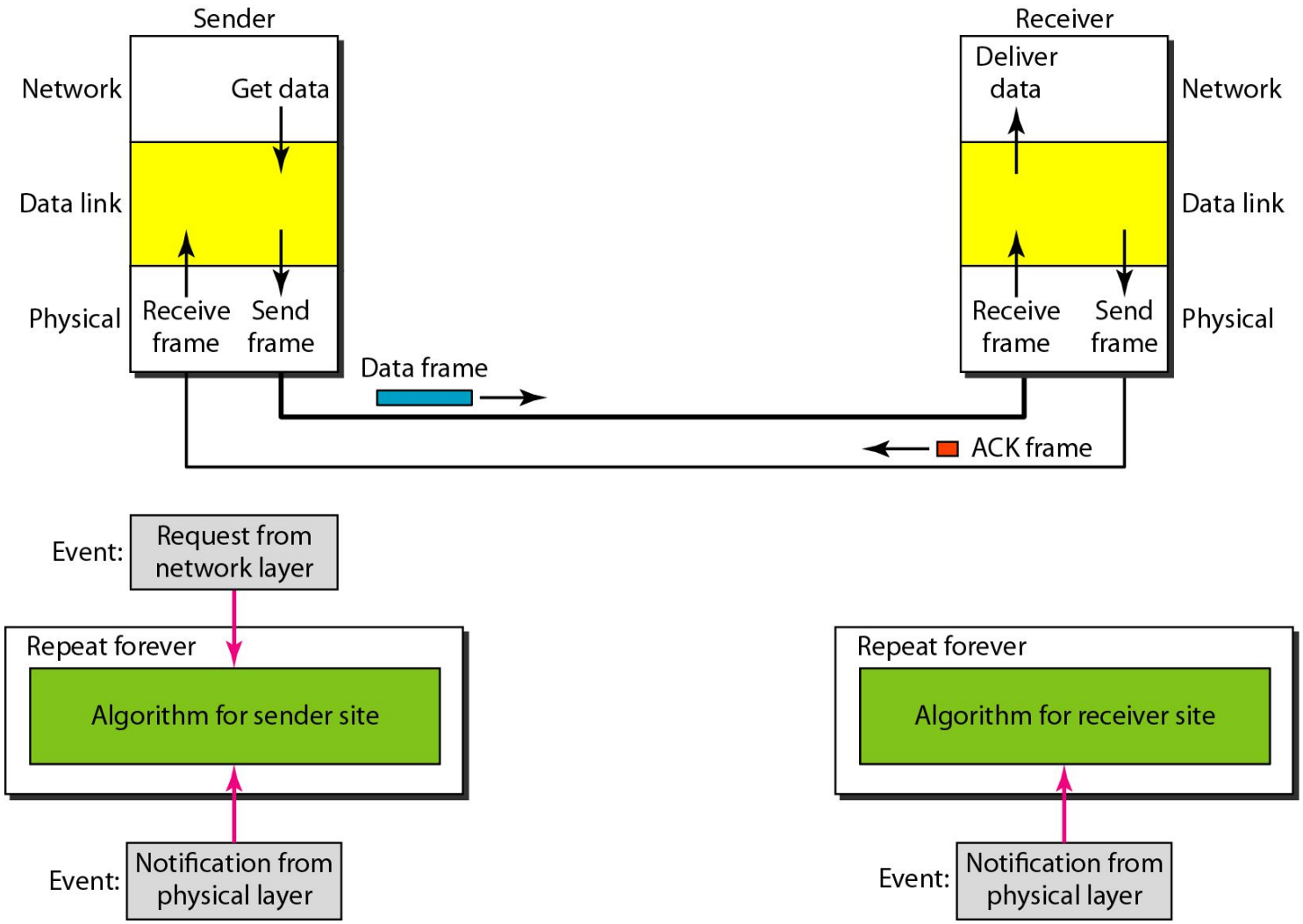


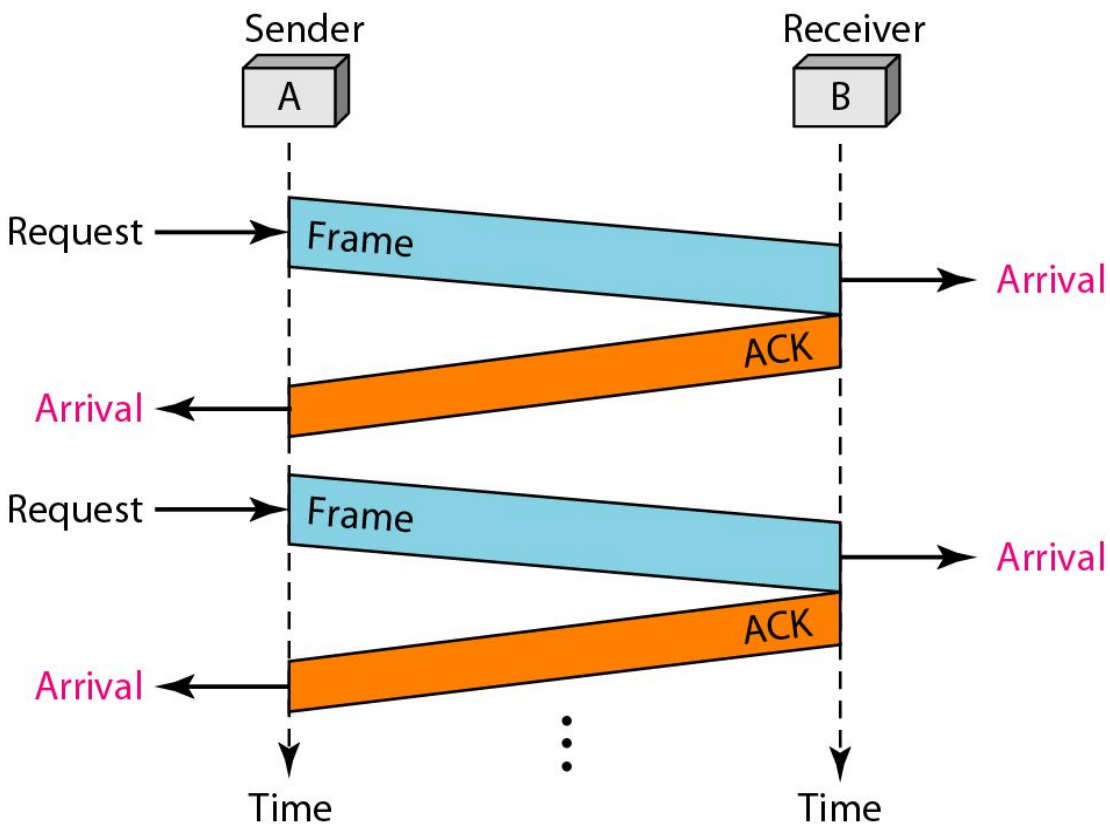
Figure 11.8 *Design of Stop-and-Wait Protocol*



Example 11.2

Figure 11.9 shows an example of communication using this protocol. It is still very simple. The sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame.

Figure 11.9 *Flow diagram for Example 11.2*



11-5 NOISY CHANNELS

Although the Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent. We discuss three protocols in this section that use error control.

Topics discussed in this section:

Stop-and-Wait Automatic Repeat Request
Go-Back-N Automatic Repeat Request
Selective Repeat Automatic Repeat Request

Note

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

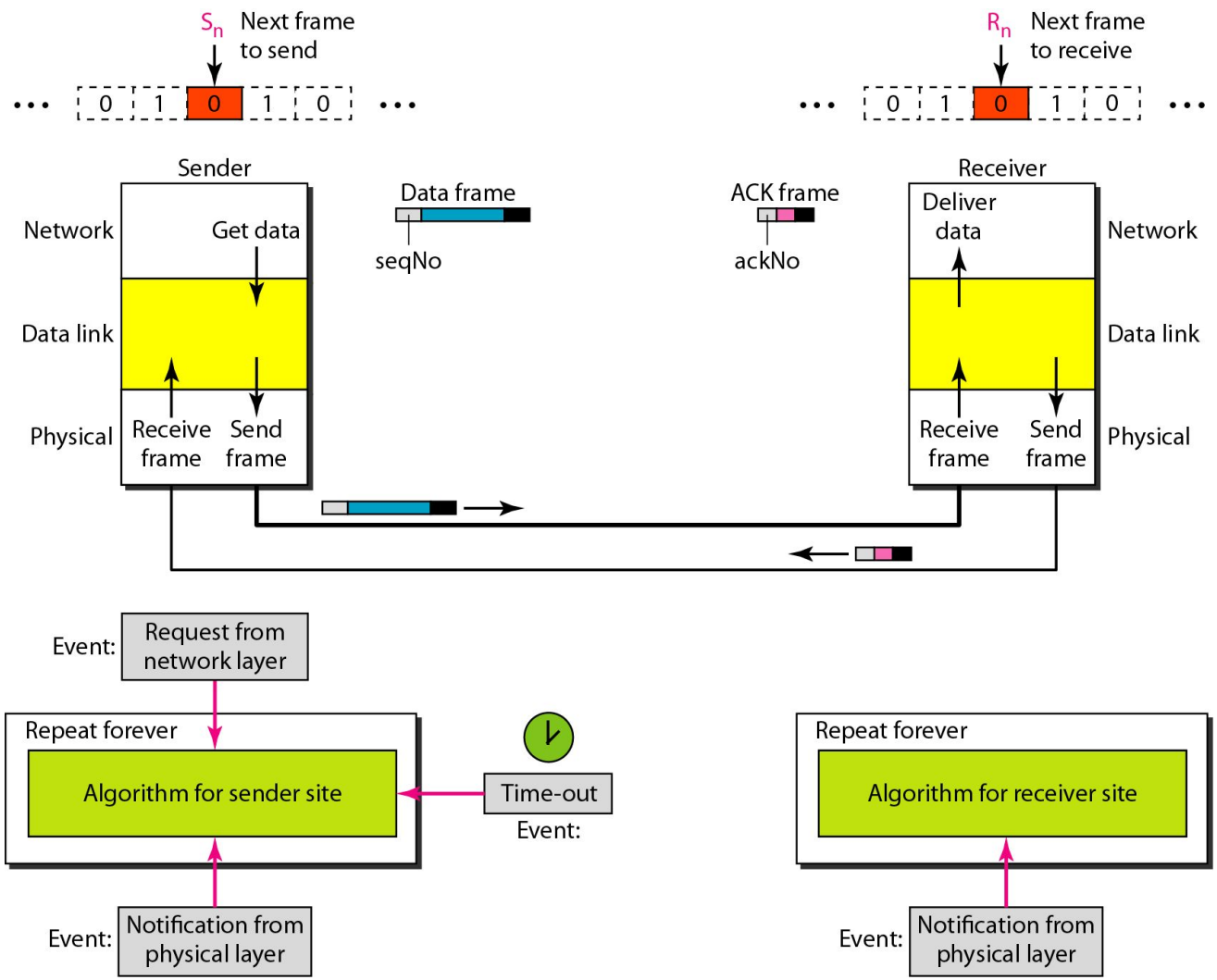
Note

In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic.

Note

In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

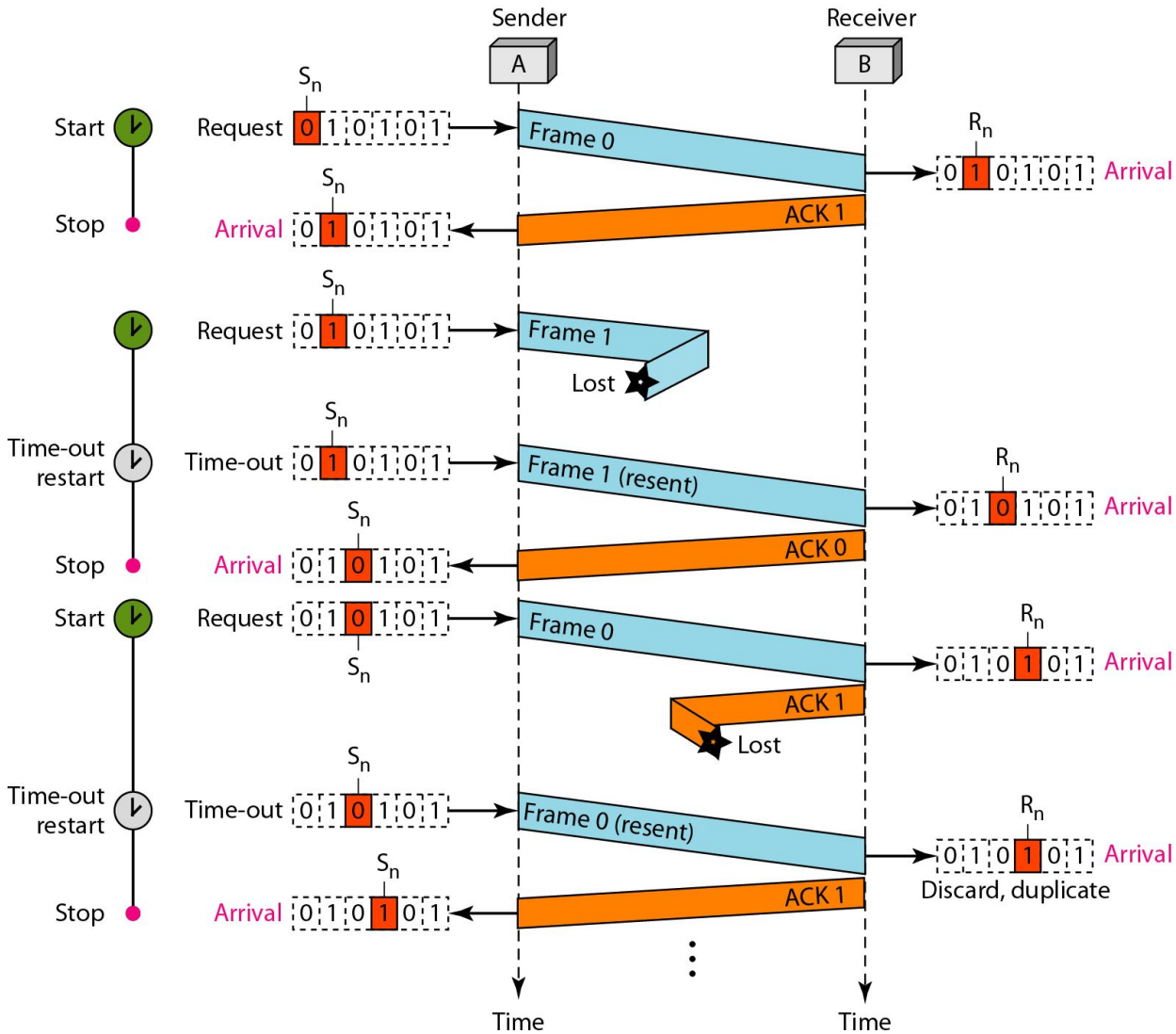
Figure 11.10 *Design of the Stop-and-Wait ARQ Protocol*



Example 11.3

Figure 11.11 shows an example of Stop-and-Wait ARQ. Frame 0 is sent and acknowledged. Frame 1 is lost and resent after the time-out. The resent frame 1 is acknowledged and the timer stops. Frame 0 is sent and acknowledged, but the acknowledgment is lost. The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged.

Figure 11.11 *Flow diagram for Example 11.3*



Sliding Window Protocol

- Sliding Window Protocols are used in computer networks to control data flow and ensure reliable delivery between a sender and a receiver.
- Instead of sending one frame at a time and waiting, the sender can send multiple frames before waiting for acknowledgments (ACKs).
  The number of frames allowed at a time is called the window size.

👉 As acknowledgments arrive, the window "slides" forward, allowing new frames to be sent.

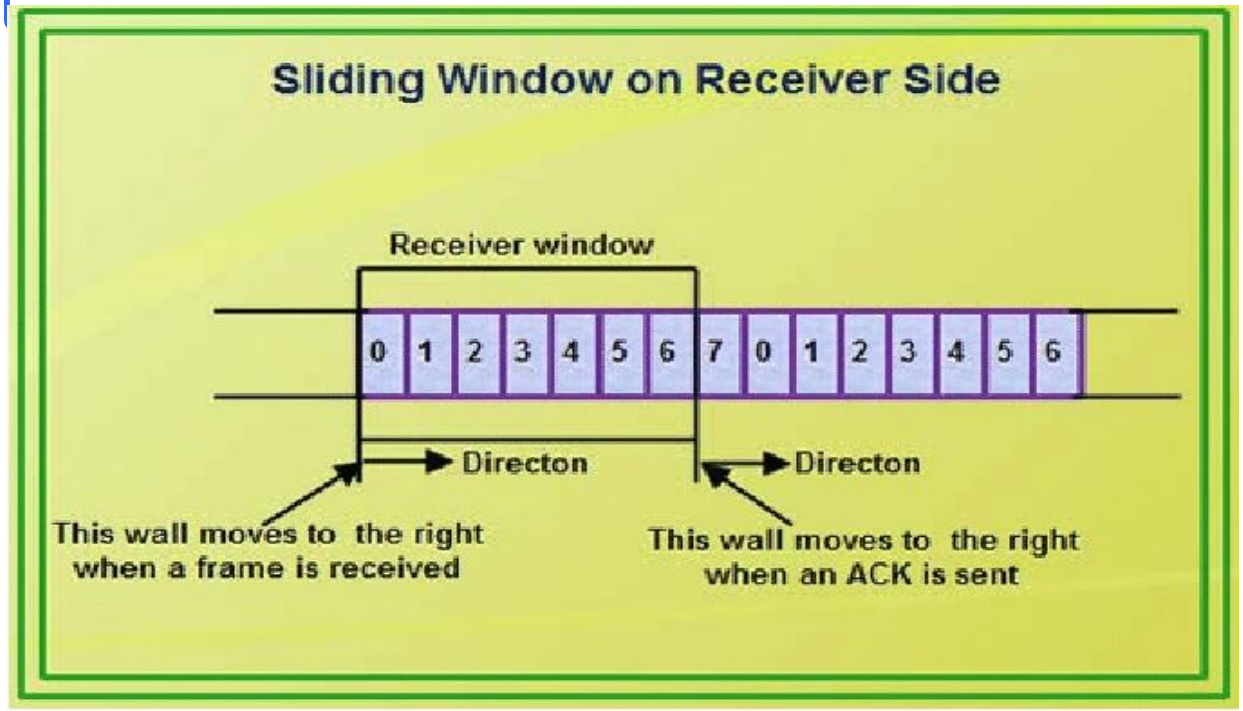
Why Sliding Window?

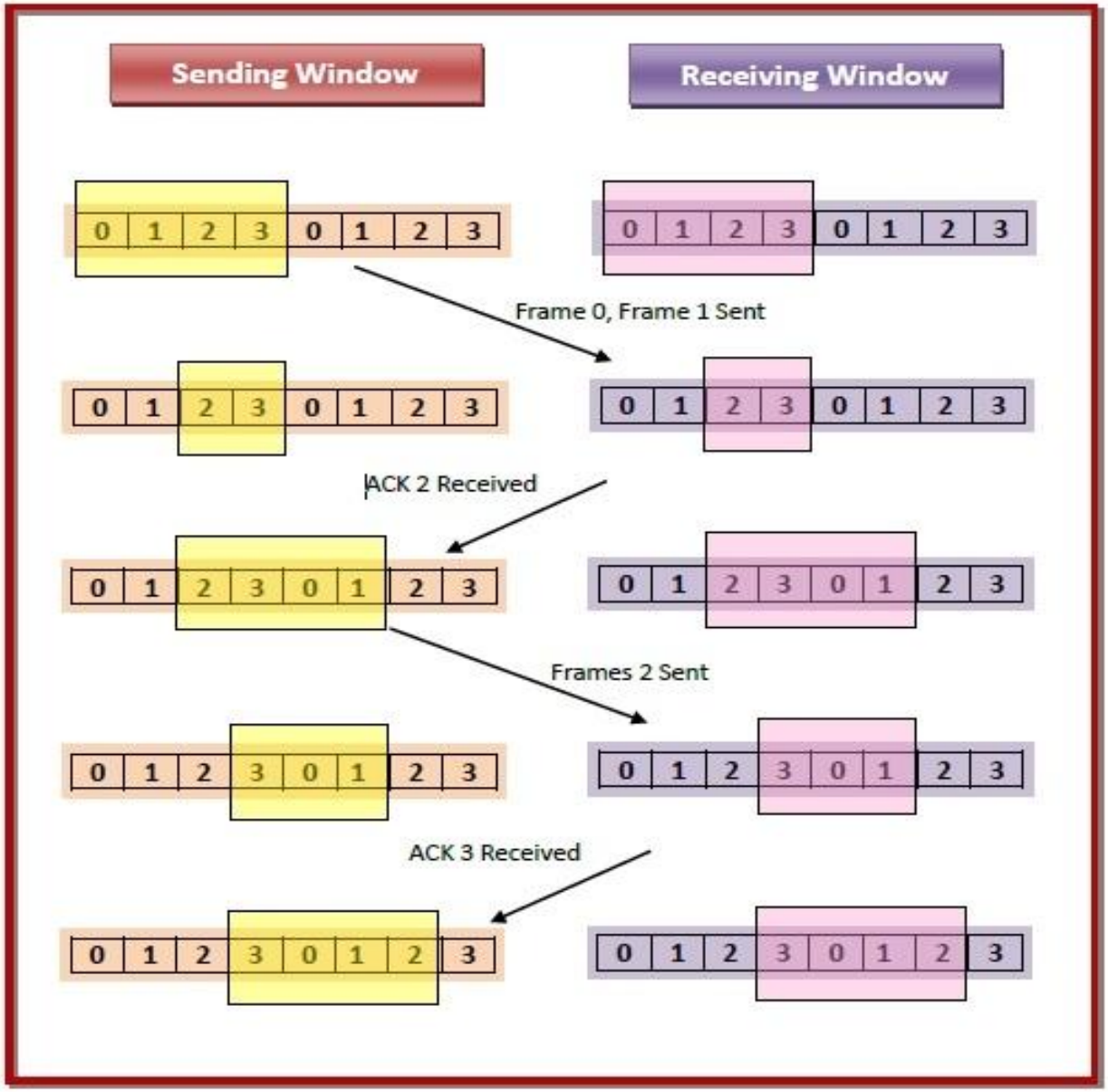
- Efficient use of bandwidth
- Flow control (don't overwhelm receiver)
- Error control (handle lost/damaged frames)

What is a Sliding Window?

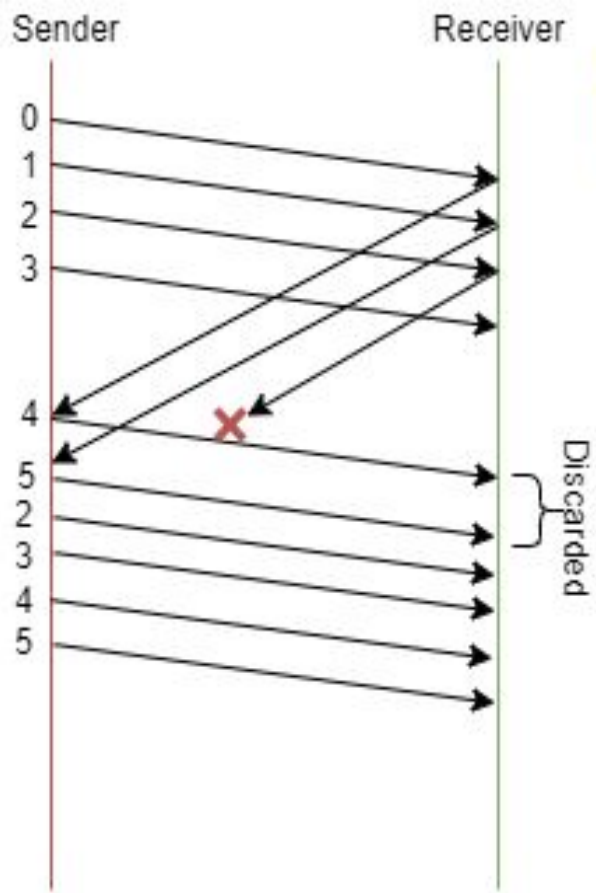
- Window = range of frame numbers
- Sender can send frames within window
- Receiver acknowledges received frames
- Window slides forward on ACK

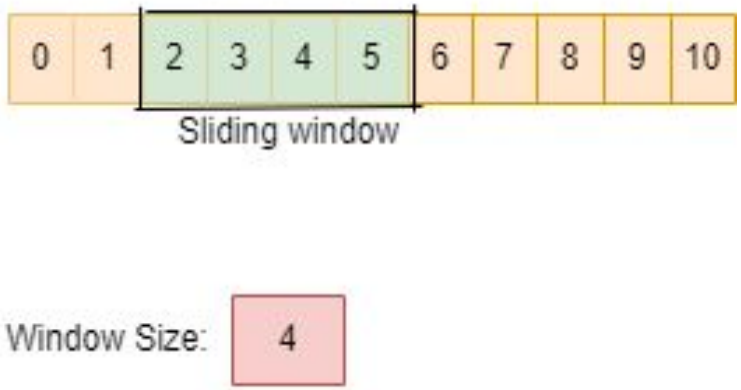
"Send several frames, wait for ACKs, slide the window forward, repeat."




Sending Window
Receiving Window
0 1 2 3 0 1 2 3
0 1 2 3 0 1 2 3
Frame 0, Frame 1 Sent
0 1 2 3 0 1 2 3
0 1 2 3 0 1 2 3
ACK 2 Received
0 1 2 3 0 1 2 3
0 1 2 3 0 1 2 3
Frames 2 Sent
0 1 2 3 0 1 2 3
0 1 2 3 0 1 2 3
ACK 3 Received
0 1 2 3 0 1 2 3
0 1 2 3 0 1 2 3

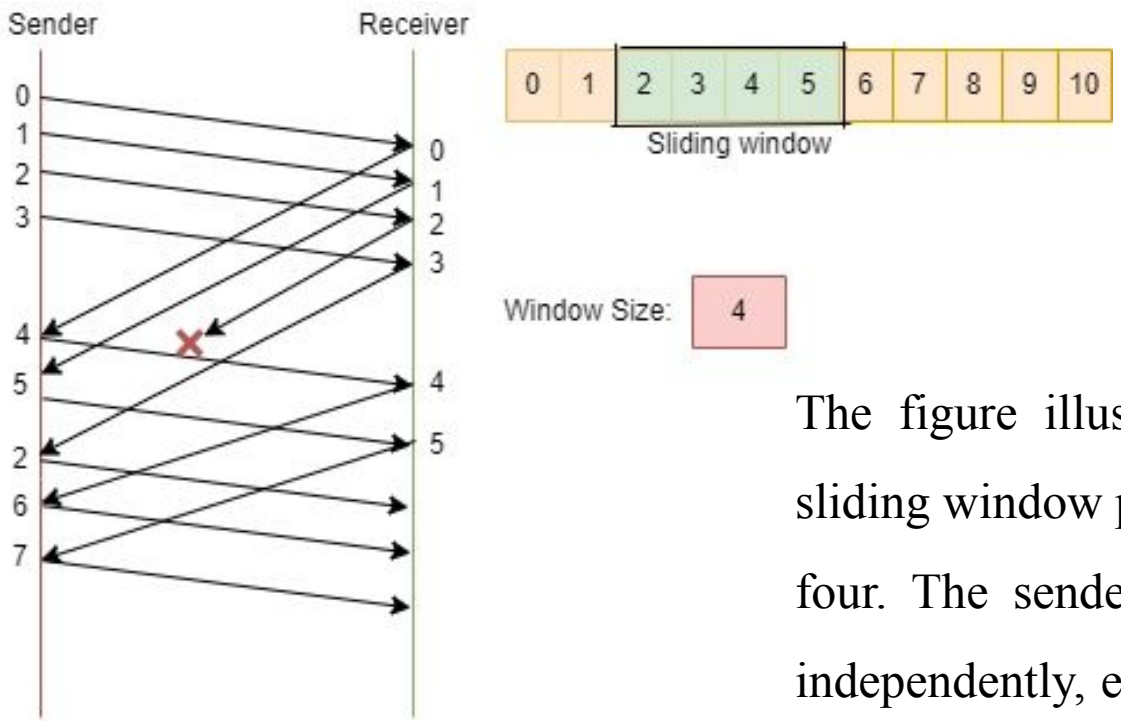
Go-Back-N





The figure illustrates the Go-Back-N sliding window protocol with a window size of four. The sender transmits multiple frames without waiting for acknowledgments. When a frame or its acknowledgment is lost, the receiver discards all subsequent out-of-order frames. Since the receiver does not buffer frames, it continues to acknowledge only the last correctly received frame. Upon

Selective Repeat



The figure illustrates the Selective Repeat sliding window protocol with a window size of four. The sender transmits multiple frames independently, each with its own timer. When a frame is lost, the receiver continues to accept and buffer subsequent out-of-order frames instead of discarding them. Individual acknowledgments are sent for each correctly received frame. Upon timeout, only the

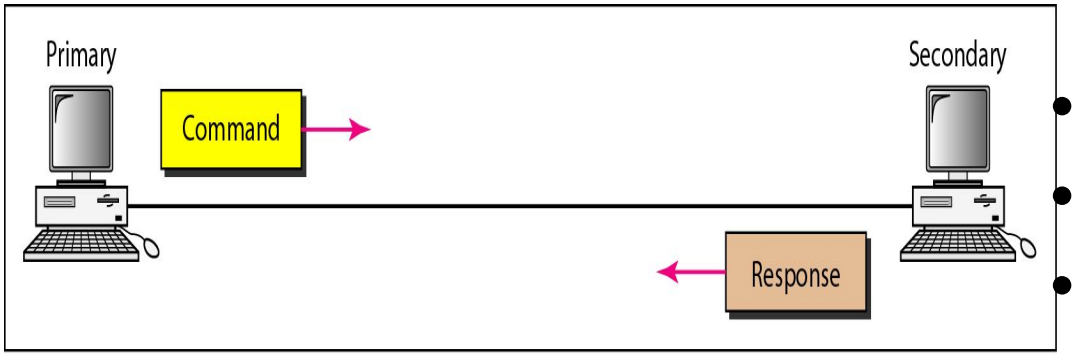
11-6 HDLC

High-level Data Link Control (HDLC) ***is a*** ***bit-oriented*** ***protocol for communication over point-to-point and multipoint links. It implements the ARQ mechanisms we discussed in this chapter.***

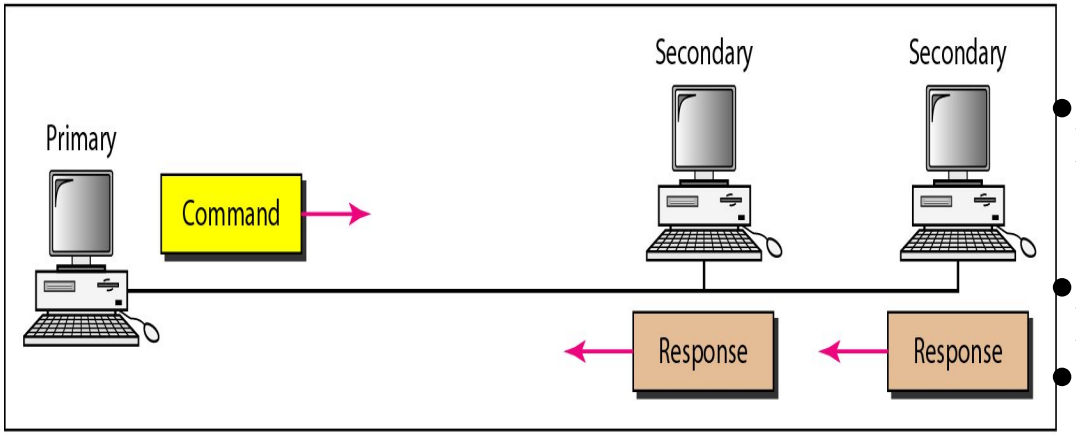
Topics discussed in this section:

Configurations and Transfer Modes
Frames
Control Field

Figure 11.25 *Normal response mode*



a. Point-to-point



b. Multipoint

Primary Station

- Controls the link
- Initiates communication
- Sends commands

Secondary Station(s)

- Do not initiate transmission on their own
- Respond only when commanded
- Send responses

This is a master–slave (primary–secondary) relationship

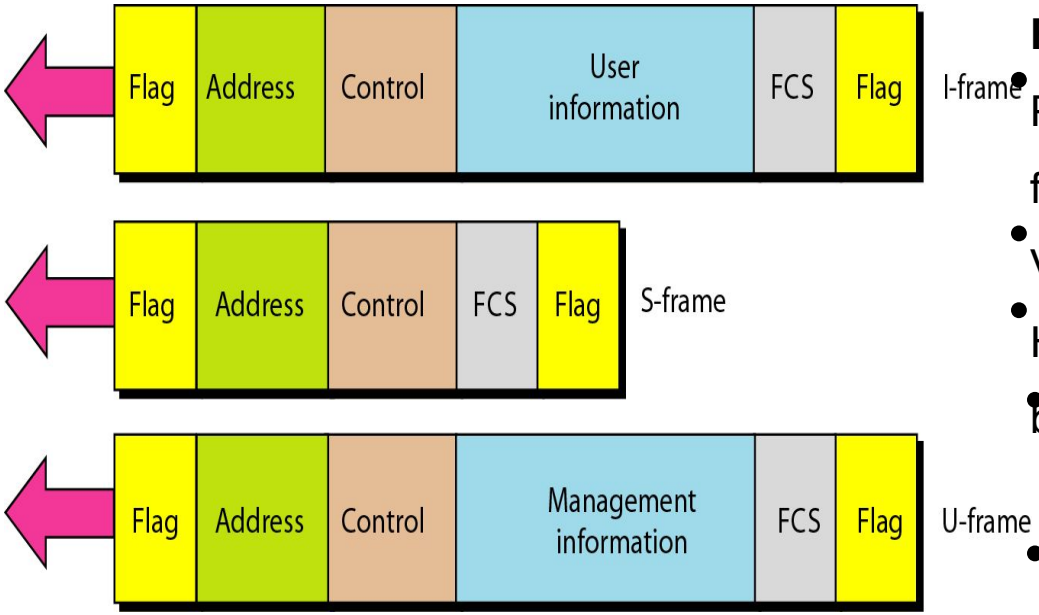
The figure shows primary–secondary communication in point-to-point and multipoint links. In point-to-point communication, a single primary communicates with a single secondary using command and response frames.

In multipoint communication, one primary controls multiple secondary stations over a shared link. The primary sends commands to one secondary at a time, and

Figure 11.26 *Asynchronous balanced mode*

The figure illustrates combined or balanced mode communication, where both stations have equal responsibility and control over the data link. Each station can transmit command or response frames without requiring permission from the other. Communication is bidirectional and symmetric, eliminating the need for polling. This mode provides higher efficiency and is commonly used in

Figure 11.27 *HDLC frames*



Flag

- Purpose: Marks the start and end of a frame
- Value: `01111110` (binary)
- Helps the receiver detect frame boundaries.

Address

- Identifies the secondary station (receiver or sender).
- Used mainly in multipoint and master–slave configurations.

Control

- Most important field
- Defines:
  - Frame type (I, S, or U)
  - Sequence numbers
  - Acknowledgments
  - Flow and error control

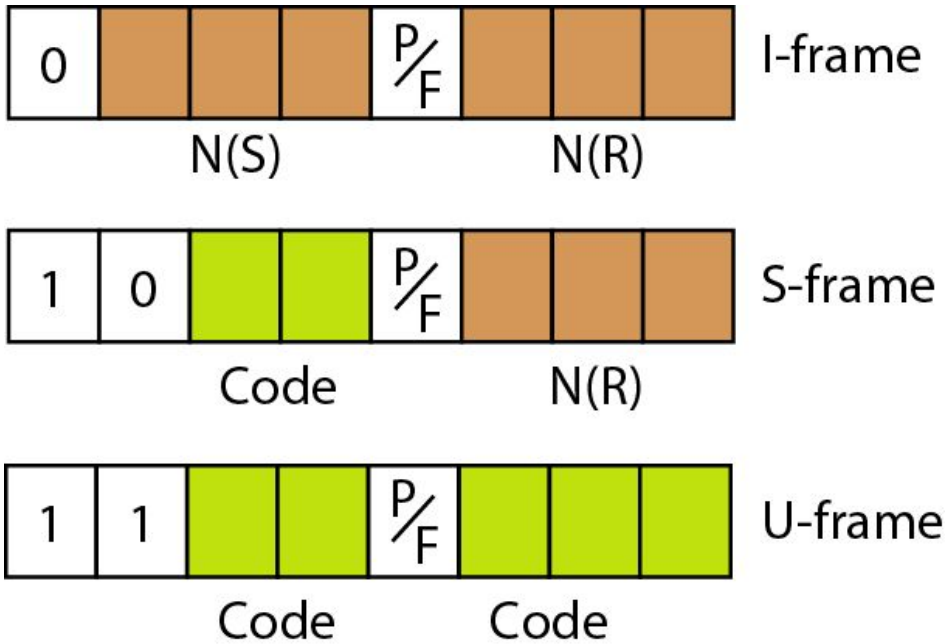
Information (Data)

- Optional
- Usually 1 byte (can be more).
- Present only in I-frames and U-frames
- Contains:
  - User data (I-frame)
  - Management/control info (U-frame)

FCS (Frame Check Sequence)

- Error detection using CRC
- Typically 16 or 32 bits
- Detects corrupted frames

Figure 11.28 *Control field format for the different frame types*



Supervisory Codes

Code	Meaning
00	RR (Receive Ready)
01	RNR (Receive Not Ready)
10	REJ (Reject – Go-Back-N)
11	SREJ (Selective Reject)

N(S) -

Sequence number of the current frame being sent.

N(R)-

Acknowledgement Number

P/F bit:

Poll (P = 1) → Sender requests immediate response

Final (F = 1) → Receiver's final response

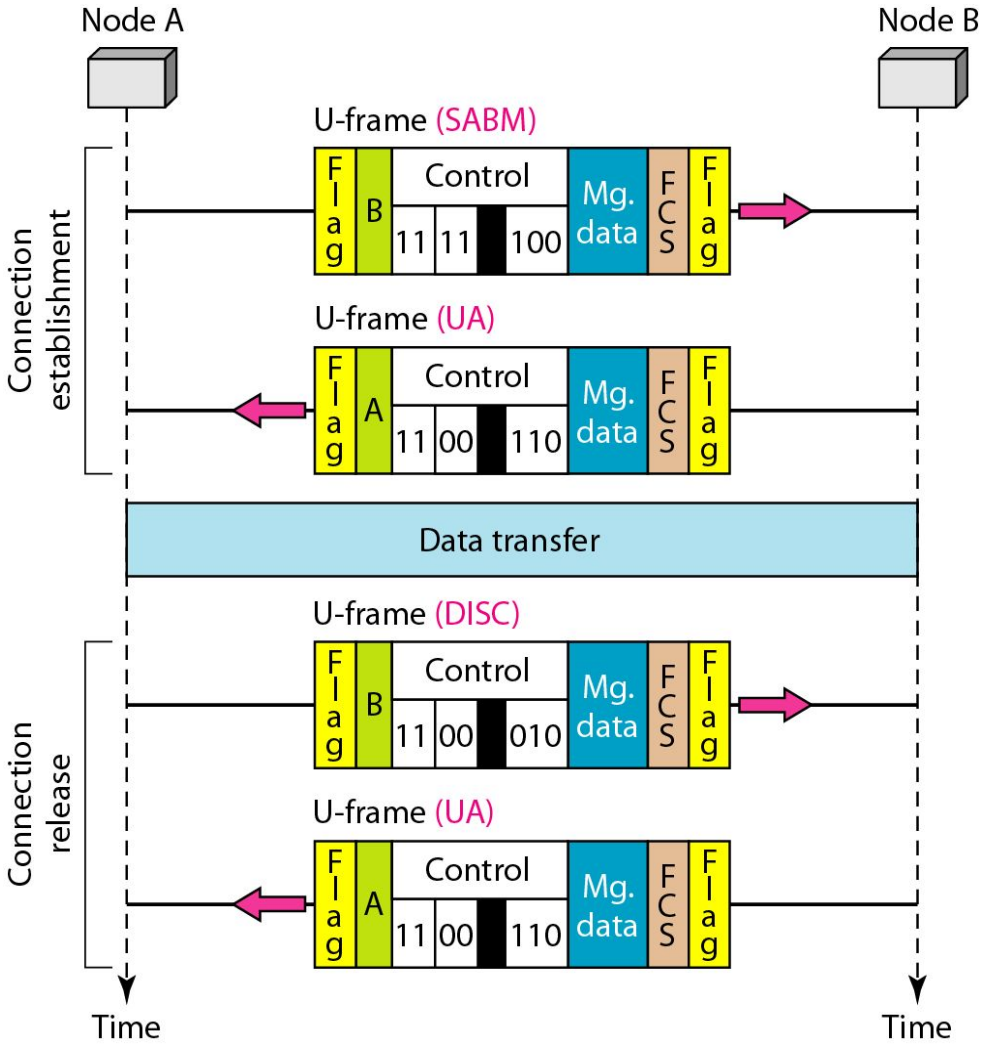
Table 11.1 *U-frame control command and response*

Code	*Command*	*Response*	*Meaning*
00 001	SNRM		Set normal response mode
11 011	SNRME		Set normal response mode, extended
11 100	SABM	**DM**	Set asynchronous balanced mode or **disconnect mode**
11 110	SABME		Set asynchronous balanced mode, extended
00 000	UI	**UI**	Unnumbered information
00 110		**UA**	**Unnumbered acknowledgment**
00 010	DISC	**RD**	Disconnect or **request disconnect**
10 000	SIM	**RIM**	Set initialization mode or **request information mode**
00 100	UP		Unnumbered poll
11 001	RSET		Reset
11 101	XID	**XID**	Exchange ID
10 001	FRMR	**FRMR**	Frame reject

Example 11.9

Figure 11.29 shows how U-frames can be used for connection establishment and connection release. Node A asks for a connection with a set asynchronous balanced mode (SABM) frame; node B gives a positive response with an unnumbered acknowledgment (UA) frame. After these two exchanges, data can be transferred between the two nodes (not shown in the figure). After data transfer, node A sends a DISC (disconnect) frame to release the connection; it is confirmed by node B responding with a UA (unnumbered acknowledgment).

Figure 11.29 *Example of connection and disconnection*



Piggybacking

- Piggybacking is a technique in full-duplex communication where an ACK (acknowledgment) is not sent as a separate frame. Instead, the ACK is carried ("piggybacked") inside the data frame going in the opposite direction.

"I'll acknowledge your data while I'm sending my own data."

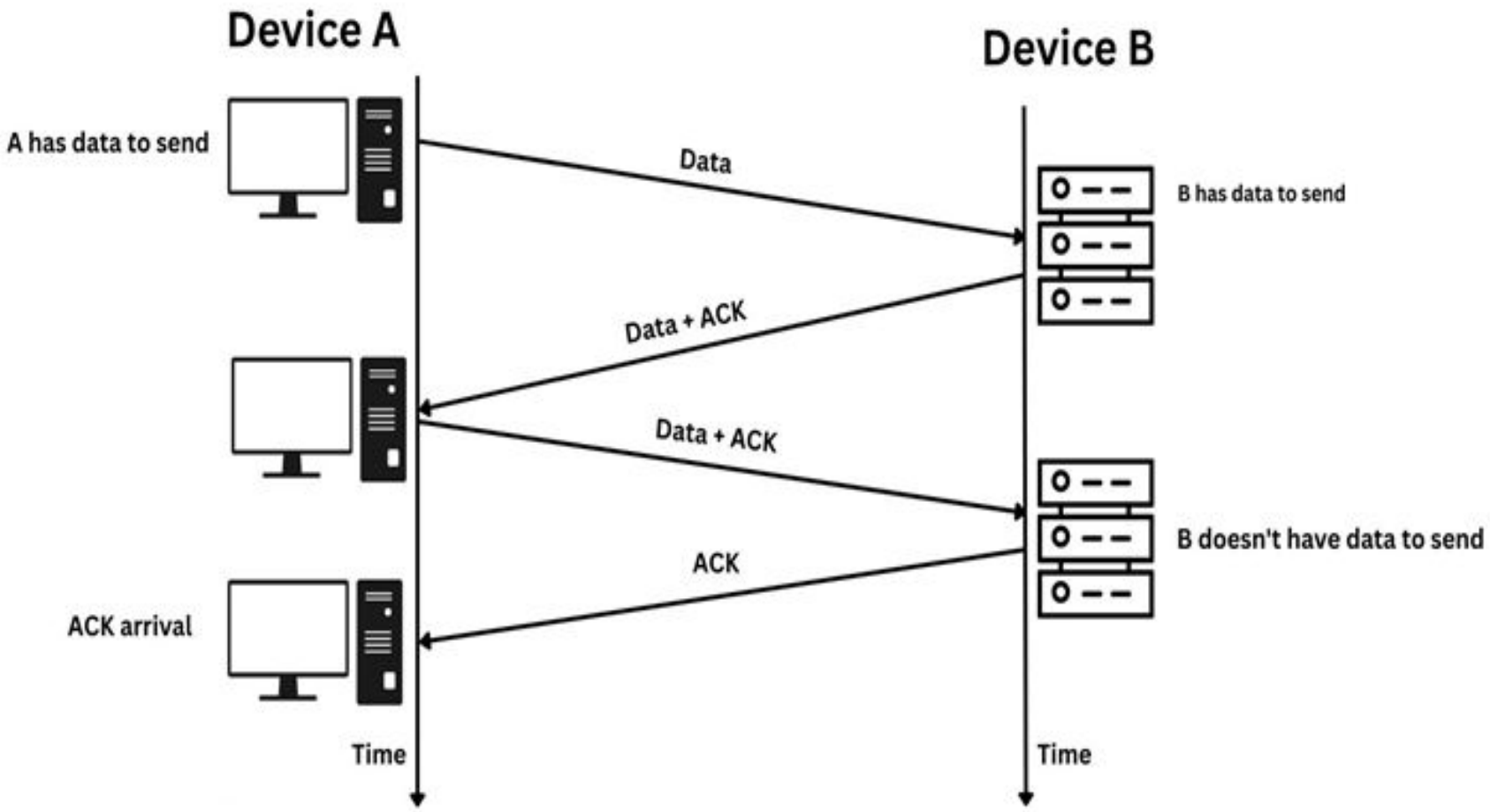
Without piggybacking:

- Data frame → ACK frame → more overhead
- More frames, more bandwidth usage

With piggybacking:

- Data + ACK combined
- Fewer frames
- Better bandwidth utilization
- Higher efficiency

Piggybacking



Error Detection and Correction

Note

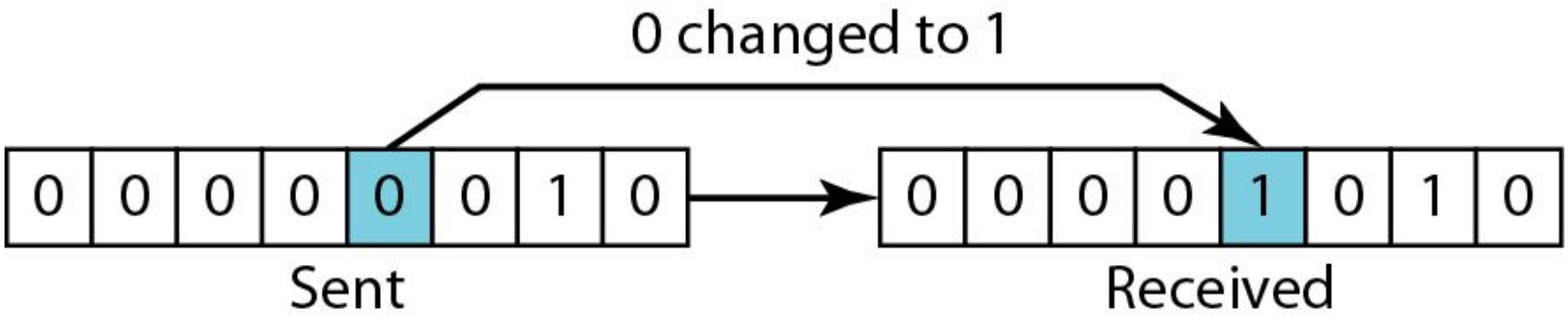
Data can be corrupted during transmission.

Some applications require that errors be detected and corrected.

Note

In a single-bit error, only 1 bit in the data unit has changed.

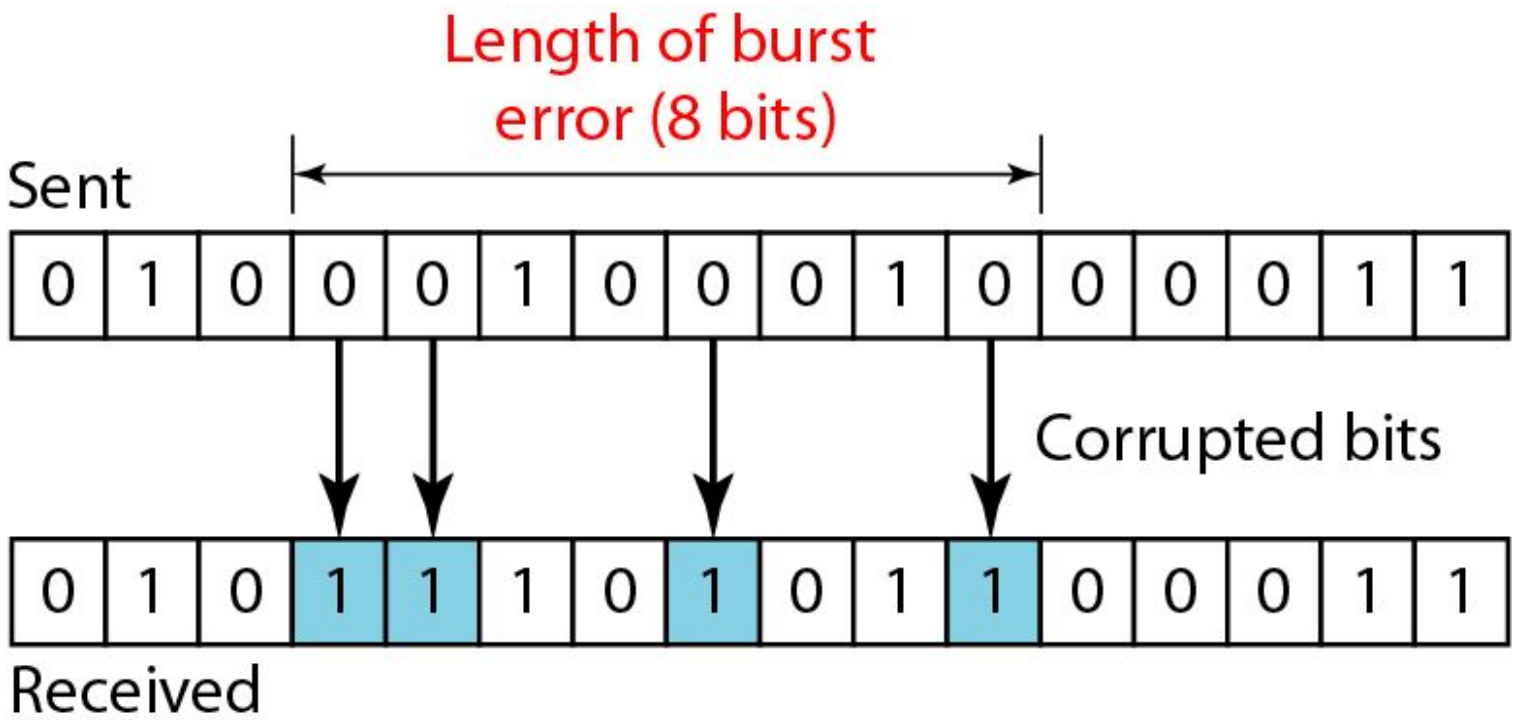
Figure 10.1 *Single-bit error*



Note

A burst error means that 2 or more bits in the data unit have changed.

Figure 10.2 *Burst error of length 8*



Note

To detect or correct errors, we need to send extra (redundant) bits with data.

Figure 10.3 *The structure of encoder and decoder*

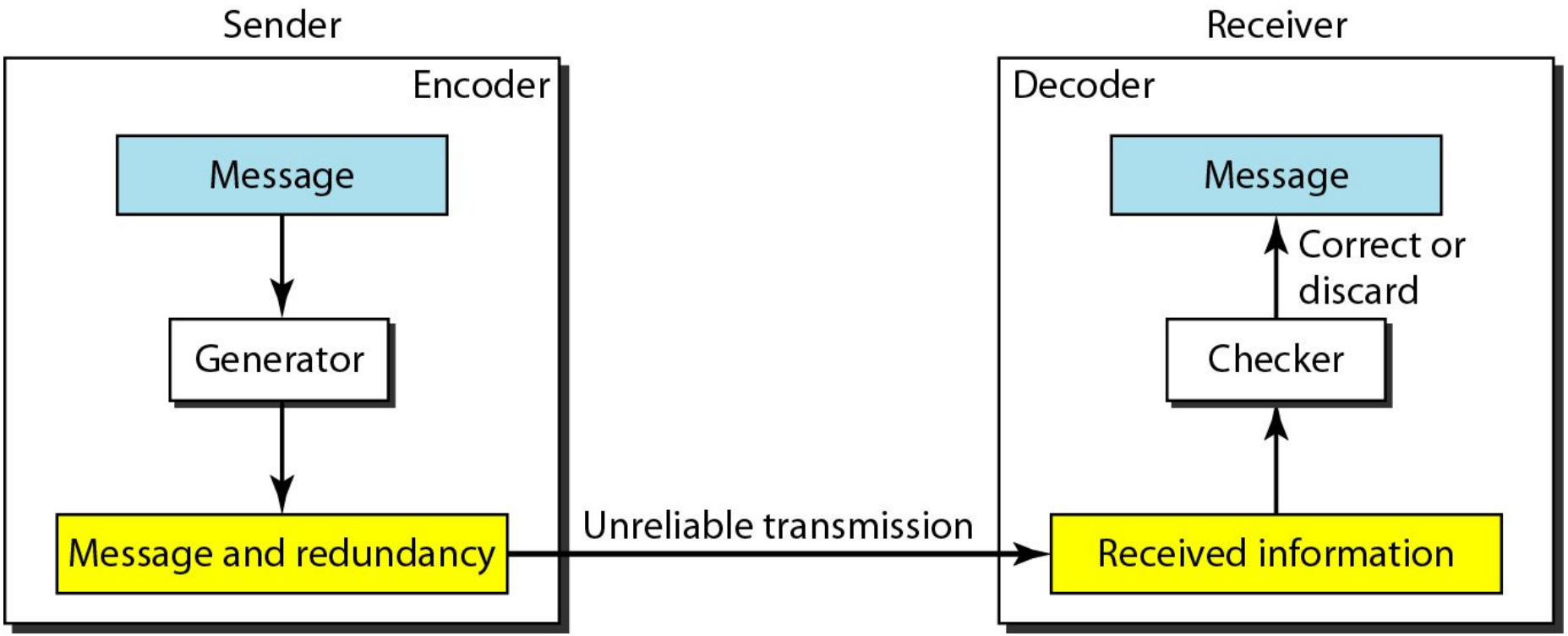


Figure 10.4 *XORing of two single bits or two words*

$0 \oplus 0 = 0$ $\qquad$ $1 \oplus 1 = 0$

a. Two bits are the same, the result is 0.

$0 \oplus 1 = 1$ $\qquad$ $1 \oplus 0 = 1$

b. Two bits are different, the result is 1.

$$
\begin{array}{cccccc}
 & 1 & 0 & 1 & 1 & 0 \\
\oplus & 1 & 1 & 1 & 0 & 0 \\
\hline
 & 0 & 1 & 0 & 1 & 0
\end{array}
$$

c. Result of XORing two patterns

10-2 BLOCK CODING

In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called codewords.

~~***Topics discussed in this section:***~~

Error Detection

Error Correction

Hamming Distance

Minimum Hamming Distance

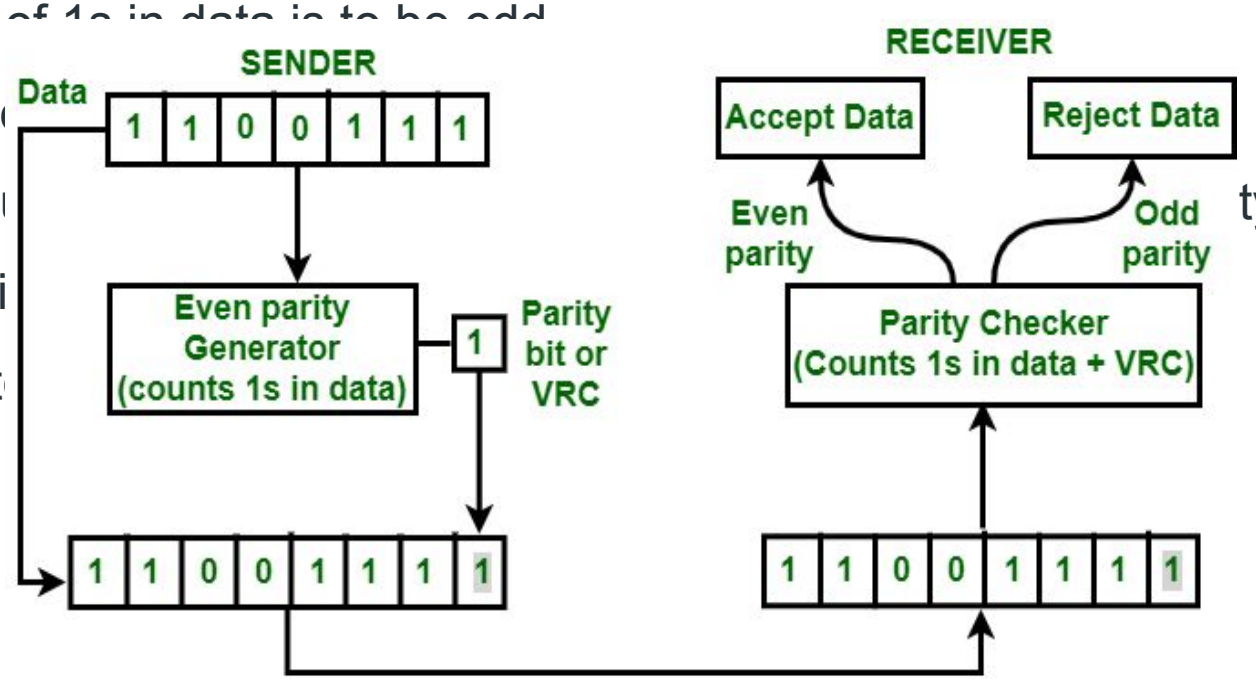
Vertical Redundancy Check(VRC)

- **Vertical Redundancy Check** is also known as Parity Check. In this method, a redundant bit also called parity bit is added to each data unit.
- This method includes even parity or odd parity. Even parity means the total number of 1s in data is to be even and odd parity means the total number of 1s in data is to be odd.

Exampl

If the so ty to the destinati

Generat

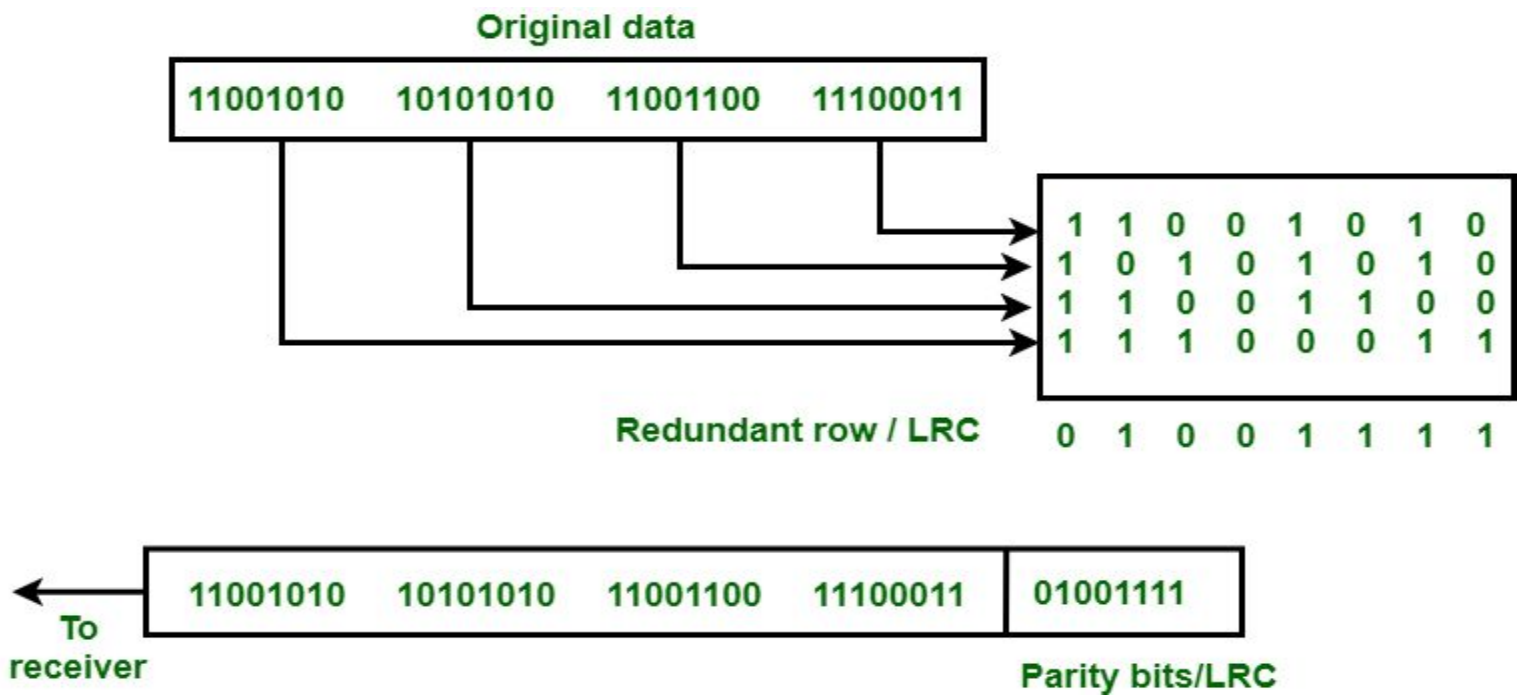


~~Longitudinal Redundancy Check (LRC)~~ is also known as 2-D parity check. In this method, data which the user want to send is organised into tables of rows and columns.

A block of bit is divided into table or matrix of rows and columns. In order to detect an error, a redundant bit is added to the whole block and this block is

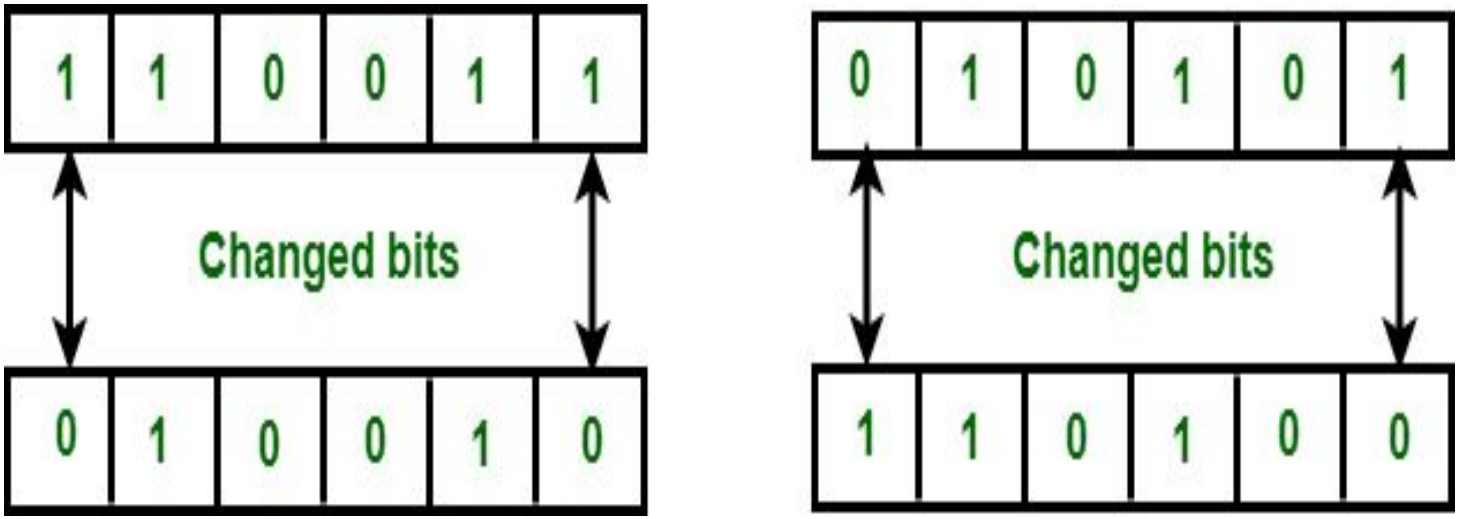
T

f



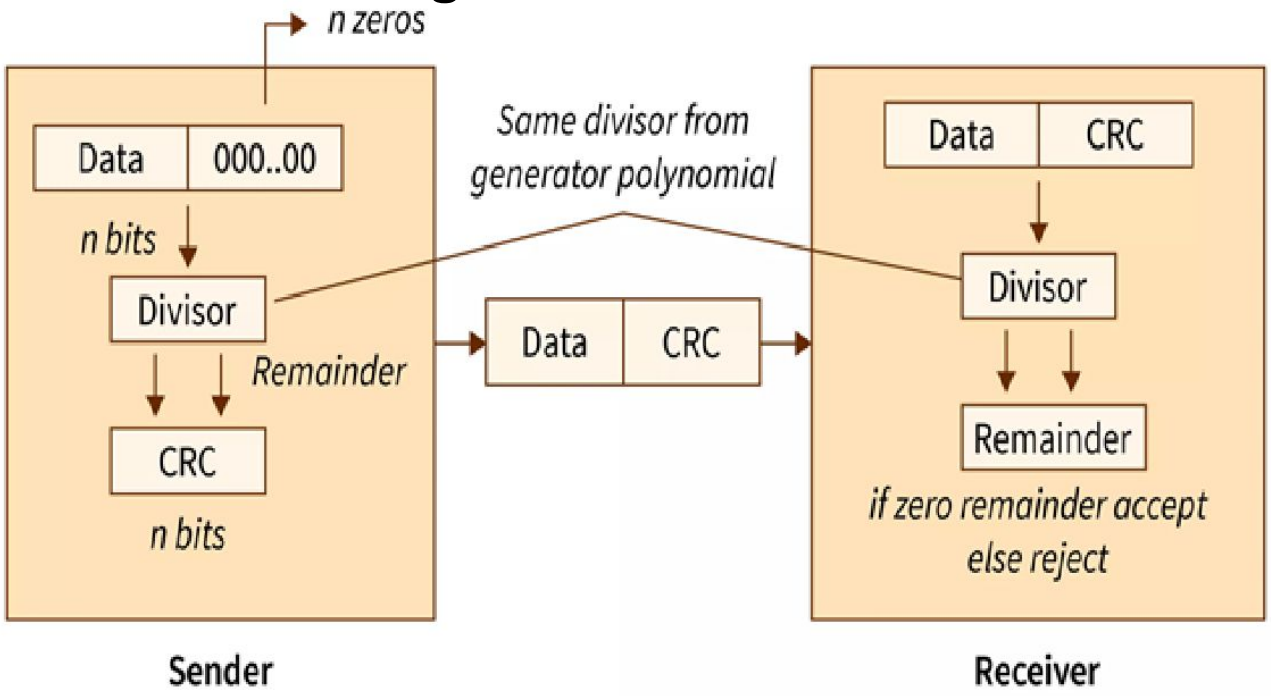
The main problem with LRC is that, it is not able to detect error if two bits in a data unit are damaged and two bits in exactly the same position in other data unit are also damaged.

Example : If data 110011 010101 is changed to 010010110100.



CRC (Cyclic Redundancy Check)

CRC is an error-detection technique that uses polynomial division to detect changes in transmitted data.



1 Sender side

Take the original data (bits).

Use a fixed rule (called a *generator polynomial*).

Perform a special kind of division (binary division).

The remainder is the CRC bits.

Send:

Data + CRC

2 Receiver side

Receive Data + CRC.

Apply the same rule.

Figure 10.15 *Division in CRC encoder*

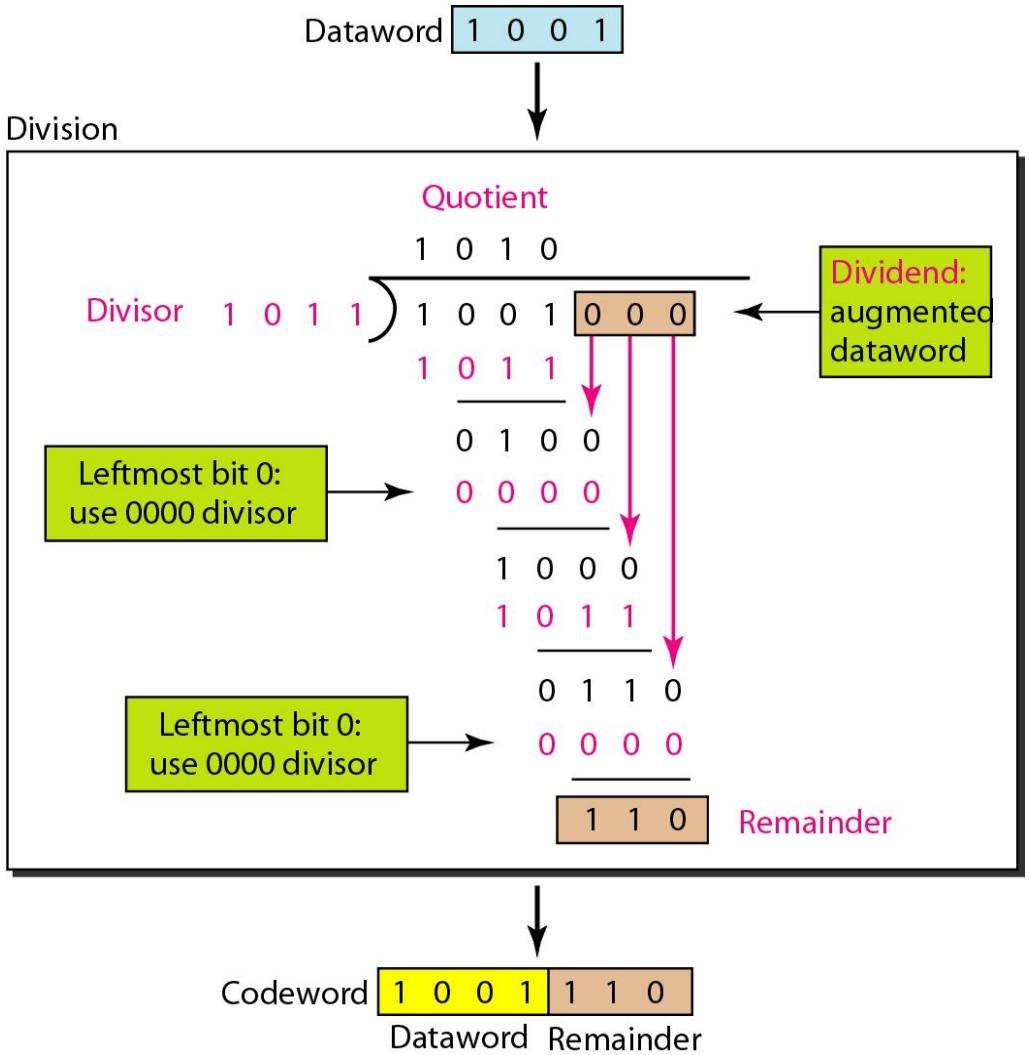
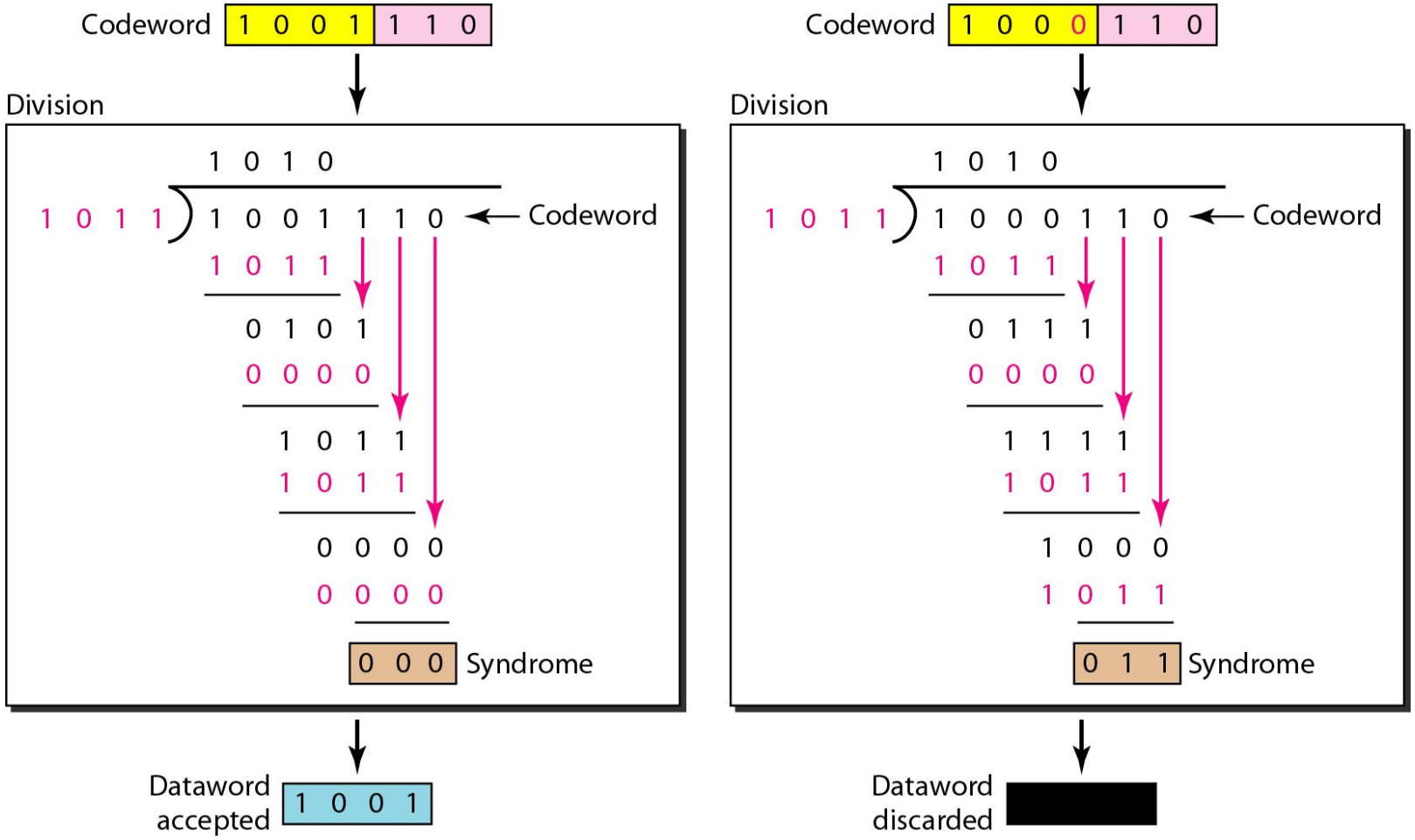


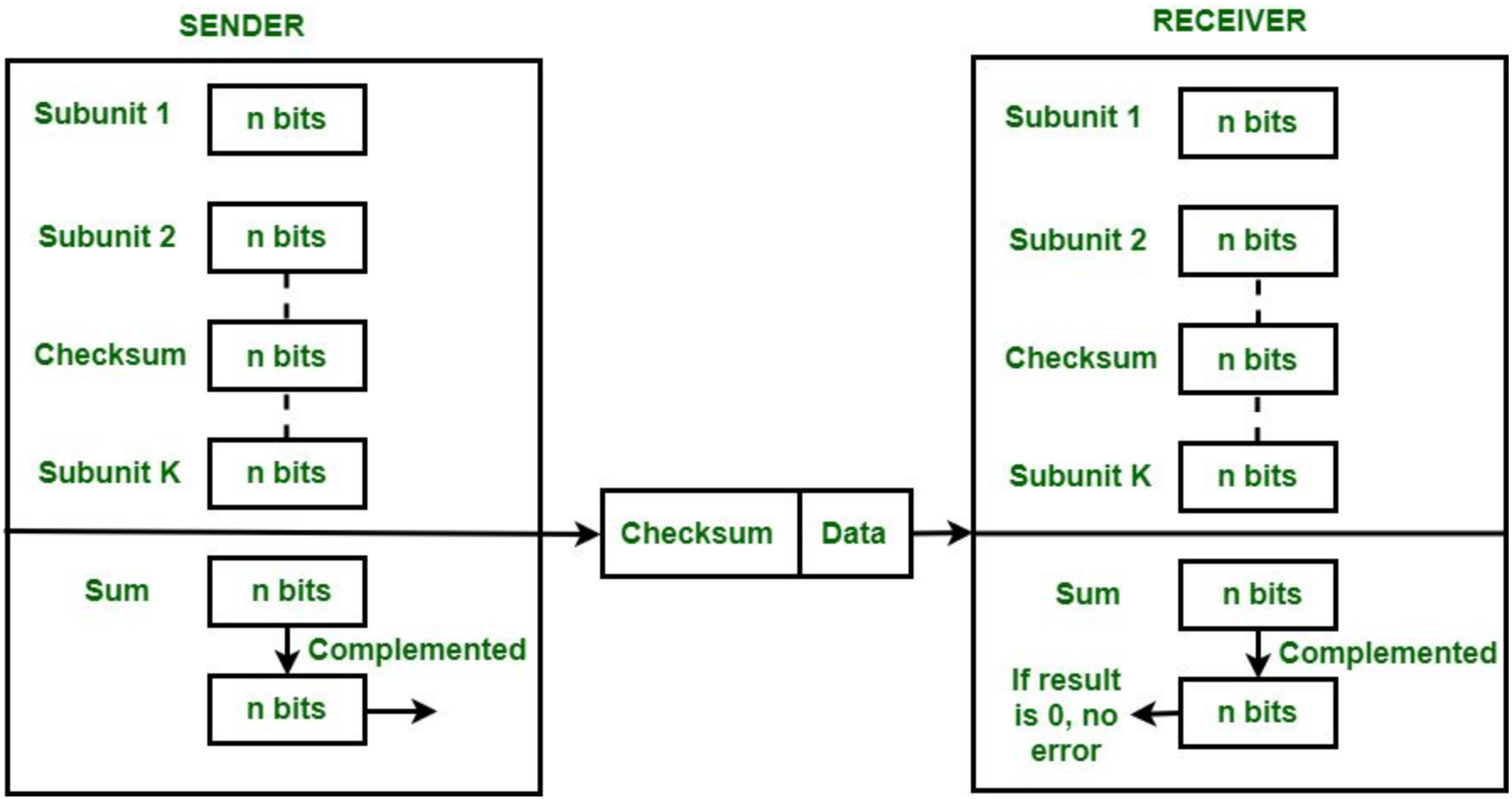
Figure 10.16 *Division in the CRC decoder for two cases*



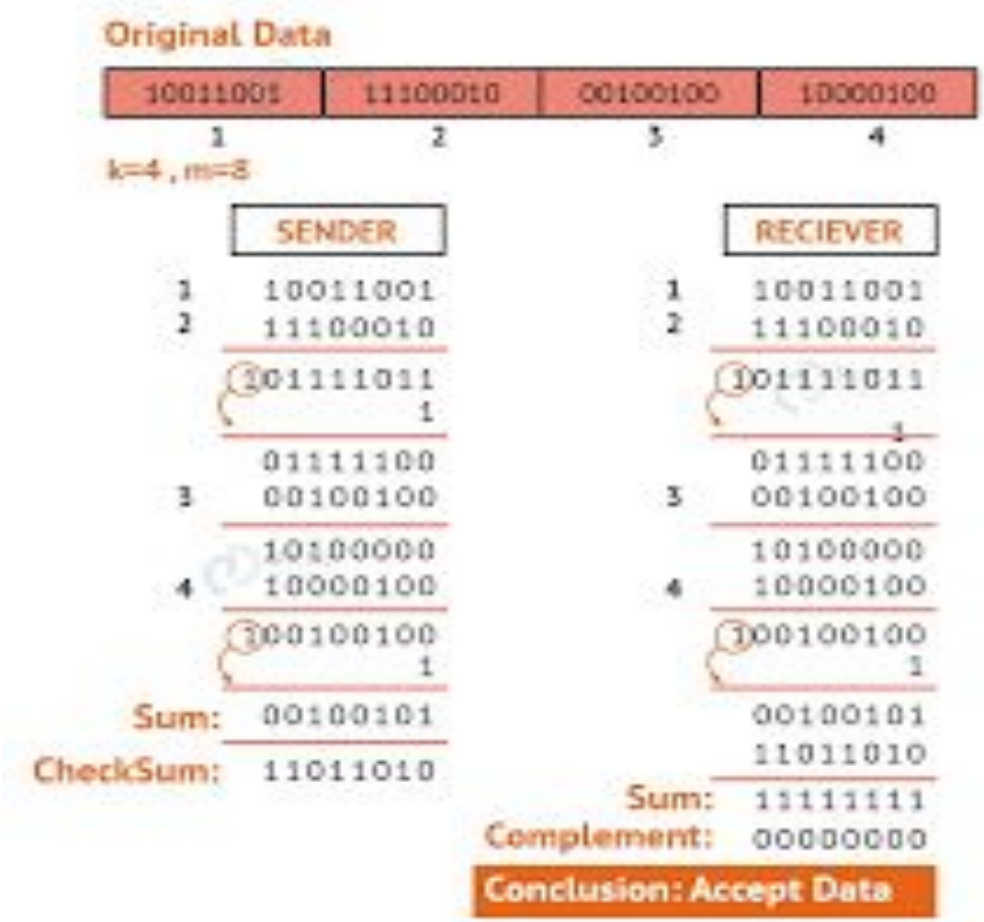
Note

The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

Checksum


SENDER
Subunit 1
n bits
Subunit 2
n bits
Checksum
n bits
Subunit K
n bits
Sum
n bits
Complemented
n bits
Checksum
Data
RECEIVER
Subunit 1
n bits
Subunit 2
n bits
Checksum
n bits
Subunit K
n bits
Sum
n bits
Complemented
If result is 0, no error
n bits

Checksum - Example



Error Correction – Hamming Code

https://www.geeksforgeeks.org/hamming-code-in-computer-network/

Multiple Access Protocols

- **ALOHA**
  - Developed in the 1970s for a packet radio network by Hawaii University.
  - **ALOHA** is a **MAC (Medium Access Control) protocol** used in shared broadcast networks where multiple nodes transmit over a common channel.
- **Types of ALOHA**

  There are **two versions**:
  - Pure ALOHA
  - Slotted ALOHA

  Pure ALOHA:

  Whenever a station has a data, it transmits. Sender finds out whether transmission was successful or experienced a collision by listening to the broadcast from the destination station. Sender retransmits after some random time if there is a collision.

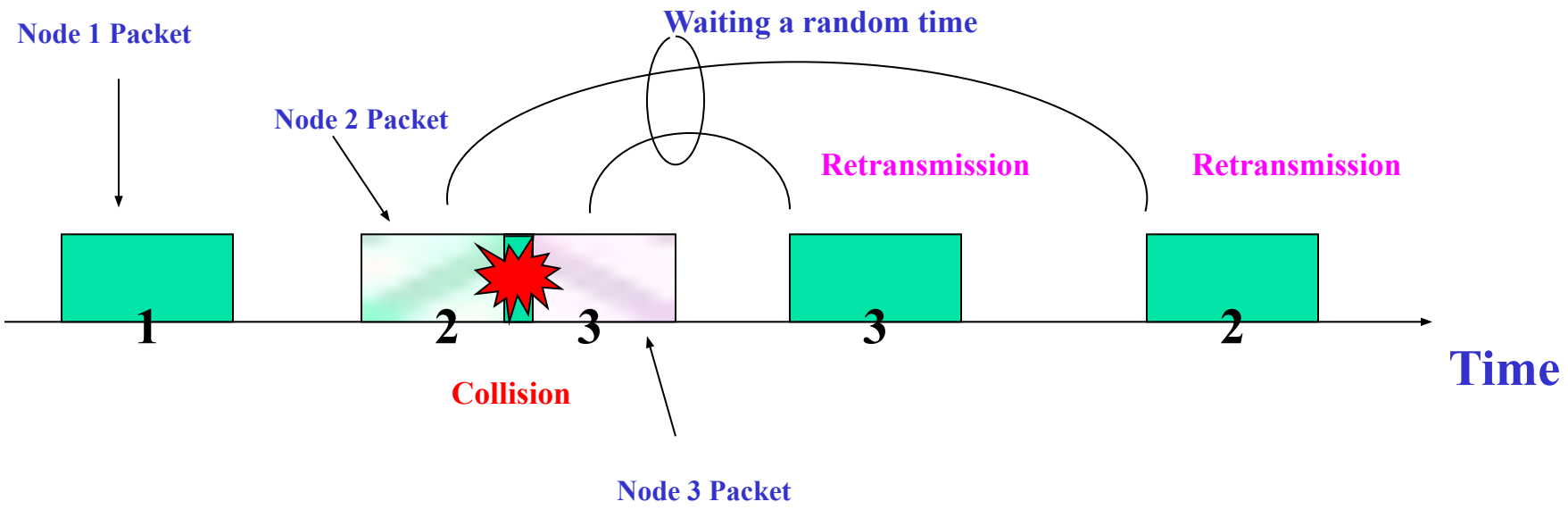
Pure ALOHA - Working Principle

- A node transmits **whenever it has data**.
- If acknowledgment (ACK) is received → Success.
- If no ACK → Collision assumed.
- Wait for **random backoff time**.
- Retransmit.

Slotted ALOHA

- Time divided into equal slots.
  - Transmission only at slot beginning.
  - Reduces vulnerable period.
  - Requires synchronization.

ALOHA



Collision mechanism in ALOHA

Throughput of ALOHA

- **The probability that *n* packets arrive in two packets time is given by**

$$P(n) = \frac{(2G)^n e^{-2G}}{n!}$$

where *G* is traffic load.

- **The probability *P(0)* that a packet is successfully received without collision is calculated by letting *n=0* in the above equation. We get**

$$P(0) = e^{-2G}$$

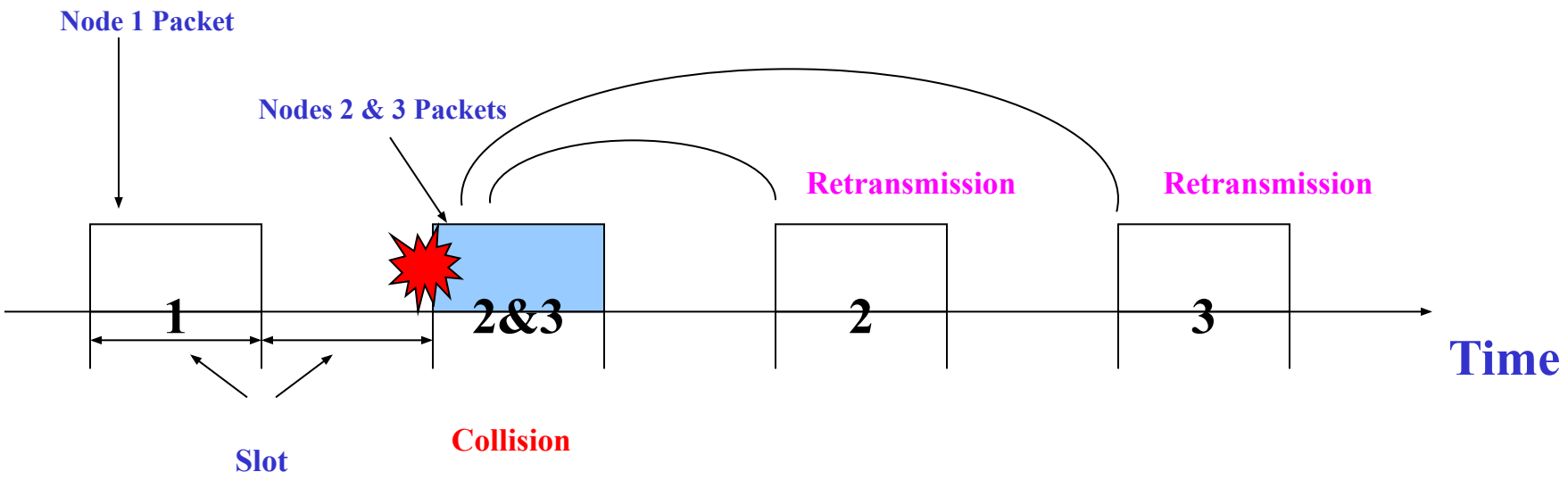
- **We can calculate throughput *S* with a traffic load *G* as follows:**

$$S = G \cdot P(0) = G \cdot e^{-2G}$$

- **The Maximum throughput of ALOHA is**

$$S_{\max} = \frac{1}{2e} \approx 0.184$$

Slotted ALOHA



Collision mechanism in slotted ALOHA

Throughput of Slotted ALOHA

- **The probability of no collision is given by**

$$P(0) = e^{-G}$$

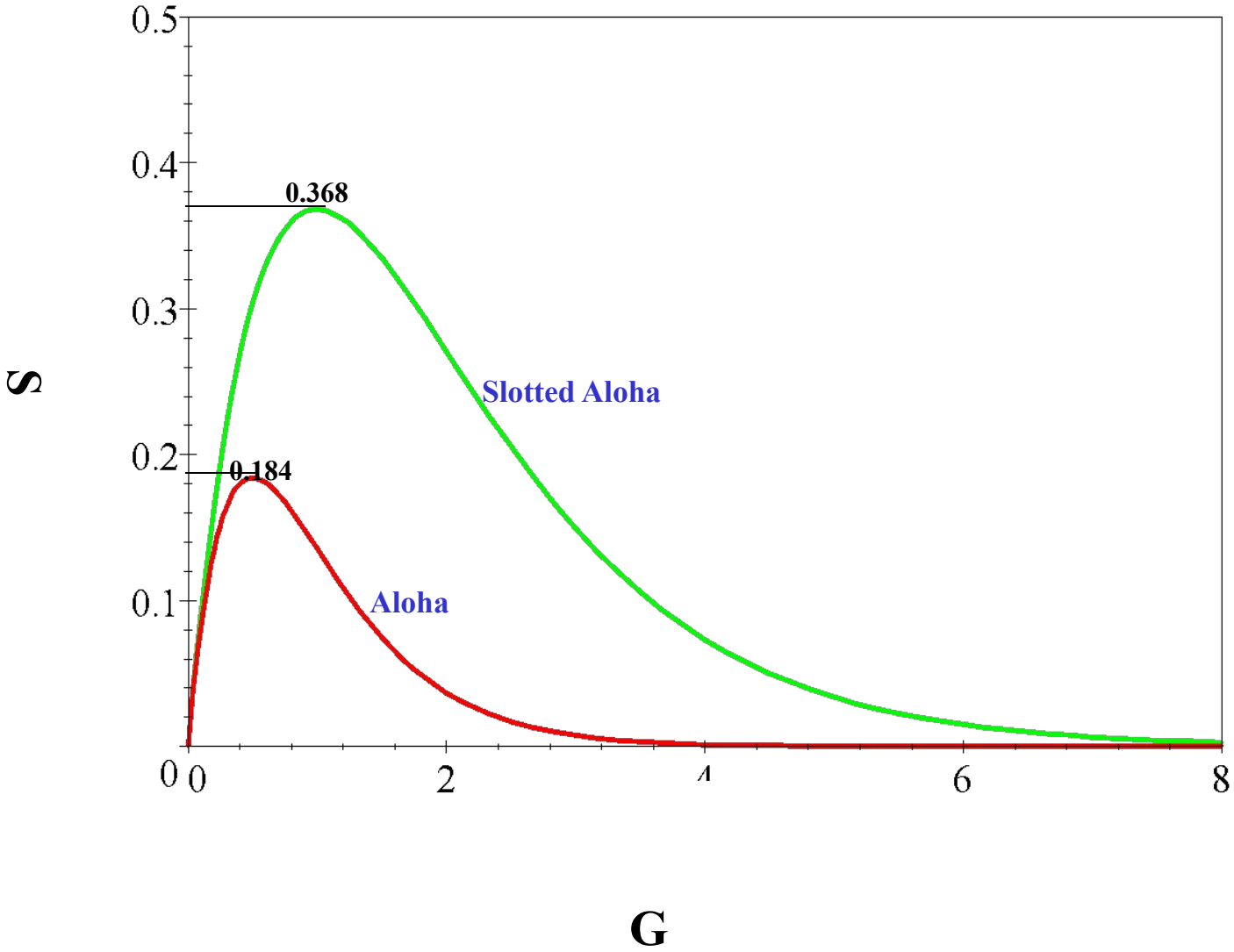
- **The throughput *S* is**

$$S = G \cdot P(0) = G \cdot e^{-G}$$

- **The Maximum throughput of slotted ALOHA is**

$$S_{\max} = \frac{1}{e} \approx 0.368$$

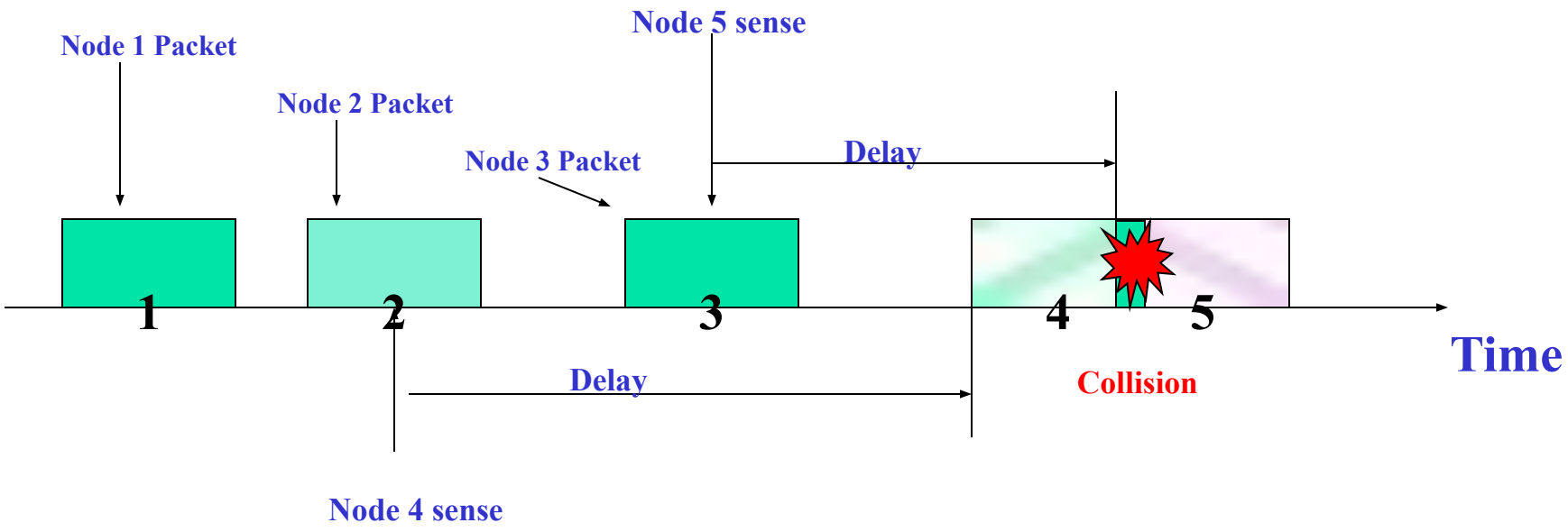
Throughput



CSMA (Carrier Sense Multiple Access)

- Max throughput achievable by slotted ALOHA is 0.368.
- CSMA gives improved throughput compared to Aloha protocols.
- Listens to the channel before transmitting a packet (avoid avoidable collisions).

Collision Mechanism in CSMA



Nonpersistent/x-persistent CSMA Protocols

- Nonpersistent CSMA Protocol:

  Step 1: If the medium is idle, transmit immediately

  Step 2: If the medium is busy, wait a random amount of time and repeat **Step 1**

  - Random backoff reduces probability of collisions
  - Waste idle time if the backoff time is too long

- 1-persistent CSMA Protocol:

  Step 1: If the medium is idle, transmit immediately

  Step 2: If the medium is busy, continue to listen until medium becomes idle, and then transmit immediately

  - There will always be a collision if two nodes want to retransmit

    (usually you stop transmission attempts after few tries)

Nonpersistent/x-persistent CSMA Protocols

- p-persistent CSMA Protocol:

  Step 1: If the medium is idle, transmit with probability p, and delay for worst case propagation delay for one packet with probability (1-p)

  Step 2: If the medium is busy, continue to listen until medium becomes idle, then go to **Step 1**

  Step 3: If transmission is delayed by one time slot, continue with **Step 1**

  - A good tradeoff between nonpersistent and 1-persistent CSMA

CSMA/CD (CSMA with Collision Detection)

- In CSMA, if 2 terminals begin sending packet at the same time, each will transmit its complete packet (although collision is taking place).
- Wasting medium for an entire packet time.
- CSMA/CD

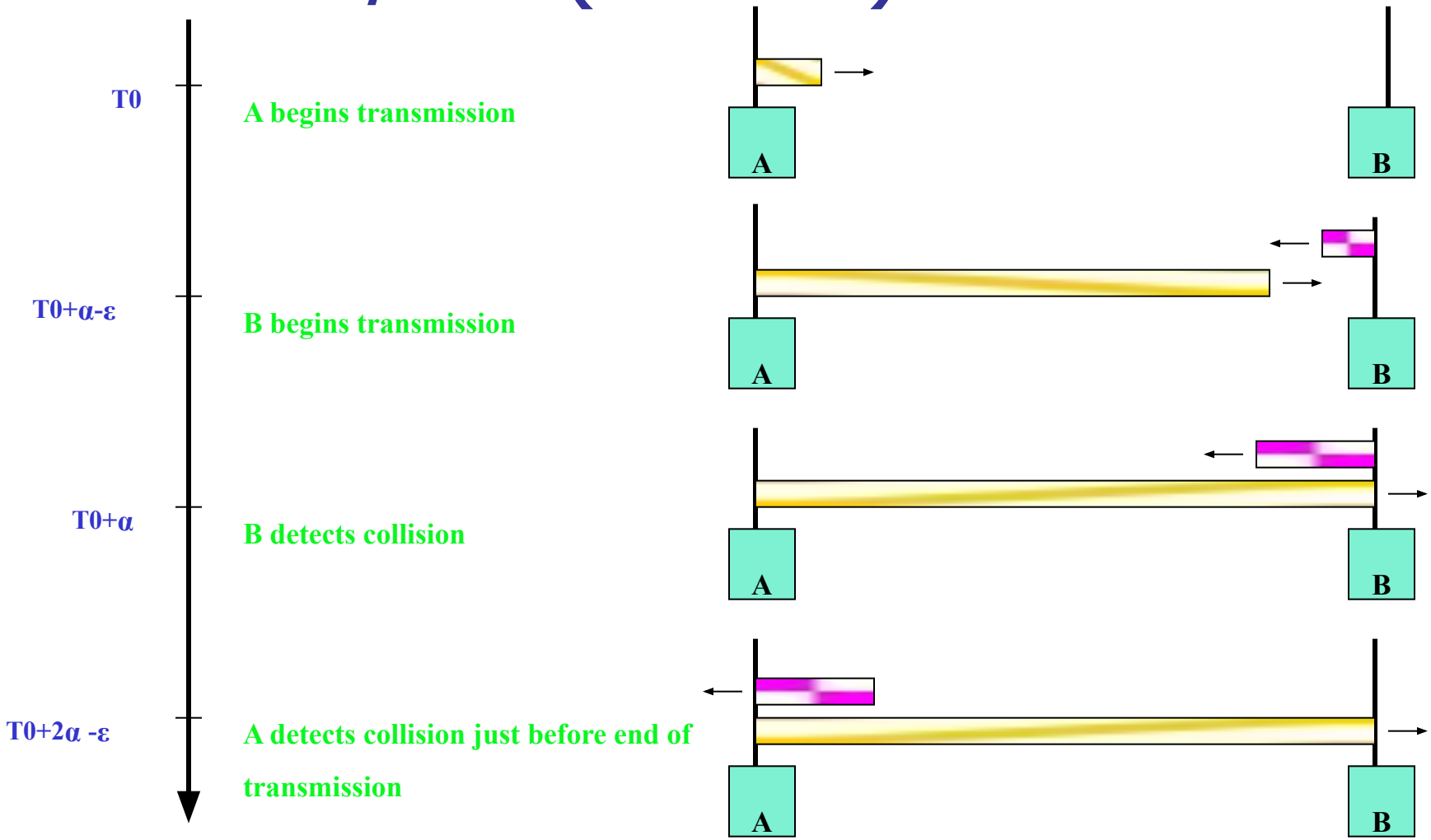
  Step 1: If the medium is idle, transmit

  Step 2: If the medium is busy, continue to listen until the channel is idle then transmit

  Step 3: If a collision is detected during transmission, cease transmitting

  Step 4: Wait a random amount of time and repeats the same algorithm

CSMA/CD (Cont'd)



(α is the propagation time)